University of Chinese Academy of Sciences

# 硕士学位论文

## 基于 MIPS 平台的 Java 虚拟机性能优化方法研究


作者姓名：陈昊轩

指导教师：齐子初　高级工程师

中国科学院计算技术研究所


学位类别：工学硕士

学科专业：计算机系统结构

培养单位：中国科学院计算技术研究所

2018 年 6 月

中国科学院计算技术研究所 鄢玺 2025-03-14 14:51:13

# Research about Performance Optimization Method of Java Virtual Machine Based on MIPS Platform

**A Thesis submitted to**

**University of Chinese Academy of Sciences**

**in partial fulfillment of the requirement**

**for the degree of**

**Master of Science**

**in Engineering**

**By**

**Haoxuan Chen**

**Supervisor： Professor Zichu Qi**

**Institute of Computing Technology, Chinese Academy of Sciences**

**June 2018**

中国科学院计算技术研究所 鄢玺 2025-03-14 14:51:13

## 中国科学院大学

## 研究生学位论文原创性声明

本人郑重声明：所呈交的学位论文是本人在导师的指导下独立进行研究工作所取得的成果。尽我所知，除文中已经注明引用的内容外，本论文不包含任何其他个人或集体已经发表或撰写过的研究成果。对论文所涉及的研究工作做出贡献的其他个人和集体，均已在文中以明确方式标明或致谢。

作者签名：

日　　期：

## 中国科学院大学

## 学位论文授权使用声明

本人完全了解并同意遵守中国科学院有关保存和使用学位论文的规定，即中国科学院有权保留送交学位论文的副本，允许该论文被查阅，可以按照学术研究公开原则和保护知识产权的原则公布该论文的全部或部分内容，可以采用影印、缩印或其他复制手段保存、汇编本学位论文。

涉密及延迟公开的学位论文在解密或延迟期后适用本声明。

作者签名：　　导师签名：

日　　期：　　日　　期：

中国科学院计算技术研究所 鄢玺 2025-03-14 14:51:13

中国科学院计算技术研究所 鄢玺 2025-03-14 14:51:13

# 摘 要

自 1995 年问世以来，Java 语言因为其安全、跨平台、面向对象等优秀特性而受到人们的欢迎，在各个行业当中都得到了广泛的应用。作为 Java 程序所运行的基础平台，Java 虚拟机的性能直接决定了 Java 应用程序的表现。随着 Java 应用程序功能与复杂度的提升，对于虚拟机性能的要求越来越高。研究 Java 虚拟机的性能优化技术，无论是在理论上还是实际应用中，都具有重大意义。

本文所完成的工作与贡献有以下三个方面。

第一、分层编译在 MIPS 上的实现。Java 虚拟机主要是针对 X86、ARM 等主流处理器架构设计的。而在 MIPS 等架构上，缺少相应实现或是优化力度不足。本文针对Java虚拟机的平台兼容性问题，实现了MIPS平台上的分层编译功能。相比于分层编译开启之前，虚拟机在客户端型应用中性能提升了 33%。

第二、解释器的性能优化。针对 Java 虚拟机解释器性能低下导致应用启动缓慢的问题，本文先量化分析了解释器分派指令的运行时开销，说明了解释器指令分派这一环节是解释器的性能瓶颈之一。在此基础上，提出了超级字节码技术，有效减少了解释器指令分派的执行次数，达到了对解释器性能优化的目的。优化后的虚拟机在解释执行时有了 4.7%的平均性能提升。

第三、虚拟机参数调优。Java 虚拟机可配置的启动参数非常多，对于不同的应用场景，需要采用不同的参数配置以充分发挥虚拟机的性能，而手工寻找较优的启动参数又存在困难。本文采用 OpenTuner 这个自动调优框架，针对Java虚拟机部分启动参数进行调优，有效提升了MIPS平台虚拟机在实际应用当中的性能，lusearch 应用性能提升更是达到了 86%。

以上三个优化方案都是在龙芯平台上基于 JDK8u77-b03 虚拟机版本进行的实验验证，这些优化代码部分将提交到龙芯 OpenJDK8 版本库当中，在实际生产当中使用。

**关键词：**Java 虚拟机、分层编译、解释器、自动调优

中国科学院计算技术研究所 鄢玺 2025-03-14 14:51:13

# Abstract

Since its appearance in 1995, Java program language has been more and more popular with people, it’s widely used in various industries for its excellent security, cross-platform, and object-oriented features. As the basic platform on which Java programs run, the performance of Java virtual machines directly determines the performance of Java applications. With the increase in the functionality and complexity of Java applications, the performance requirements are getting higher and higher. Therefore, on the Loongson platform, it is of great significance to research and optimize the performance of the virtual machine.

There are three aspects to the work and contribution accomplished in this paper.

Firstly, implementation of Tiered Compilation on MIPS platform. The Java Virtual Machine is mainly designed for mainstream processor architectures such as X86 and ARM. In MIPS and other architectures, the lack of corresponding implementations or optimization efforts is insufficient. This article aims at the platform compatibility problem of the Java virtual machine and realizes the Tiered Compilation function on the MIPS platform. Tiered Compilation capabilities have improved virtual machine startup performance by 33%.

Secondly, the interpreter's performance is optimized. For the problem that the performance of the Java virtual machine interpreter leads to slow application startup, this paper first quantifies the runtime overhead of the interpreter dispatch instruction, and shows that the interpreter instruction dispatch is a performance bottleneck of the interpreter. On this basis, a super bytecode technique is proposed, which effectively reduces the number of executions of the interpreter instruction dispatch and achieves the purpose of optimizing the interpreter performance. The optimized interpreter has a 4.7% performance increase.

Thirdly, the virtual machine parameter tuning. There are many startup parameters of the Java virtual machine. For different application scenarios, different parameters

need to be configured to give full play to the performance of the virtual machine. However, it is difficult to manually find the optimal startup parameters. This article uses OpenTuner's automatic tuning framework to tune some of the startup parameters of the Java virtual machine, which effectively improves the performance of the MIPS platform virtual machine in practical applications. Some application performance improvements reach 86%.

These optimization code parts will be integrated into the Loongson OpenJDK8 repository for use in actual production.

**Keywords:** JVM, Interpreter, Tiered Compilation, Autotuning

中国科学院计算技术研究所 鄢銮 2025-03-14 14:51:13

# 目 录

第 1 章 绪论.....1
1.1 研究背景.....1
1.2 Java 虚拟机简介.....2
1.3 本文的主要内容.....4
1.4 本文的组织结构.....5
第 2 章 Java 虚拟机背景介绍.....7
2.1 Java 虚拟机基础.....7
2.1.1 运行时数据区.....8
2.1.2 类加载器.....9
2.1.3 垃圾收集器.....11
2.1.4 执行引擎.....12
2.1.5 本地方法接口.....13
2.2 Java 虚拟机常见优化方法.....14
2.2.1 解释器优化技术.....14
2.2.2 编译器优化技术.....18
2.3 Java 虚拟机标准测试集.....24
2.4 Java 虚拟机性能对比.....25
2.5 本章小结.....27
第 3 章 分层编译的实现.....29
3.1 分层编译原理.....29
3.2 虚拟机的平台兼容性问题.....30
3.3 MIPS 平台 C1 编译器实现.....31
3.3.1 C1 编译器的基本结构.....31
3.3.2 运行时数据区调整.....33
3.3.3 LIR 汇编器实现.....34
3.3.4 适配器实现.....35
3.4 错误调试实例.....36
3.5 性能评测实验.....39
3.6 本章小结.....41

第 4 章 解释器性能优化 .....43
4.1 解释器工作原理.....43
4.1.1 基于栈的指令集与基于寄存器的指令集.....43
4.1.2 Java 虚拟机的解释器 .....44
4.2 解释器性能瓶颈分析.....47
4.3 超级字节码技术.....48
4.3.1 技术原理介绍.....48
4.3.2 合成字节码的选择.....49
4.4 性能评测实验.....50
4.5 本章小结.....52
第 5 章 虚拟机参数调优 .....53
5.1 Java 虚拟机自动调优 .....53
5.1.1 自动调优介绍.....53
5.1.2 Java 虚拟机启动参数介绍 .....53
5.2 Java 虚拟机参数适配问题 .....54
5.3 OpenTuner 框架介绍 .....55
5.4 性能评测实验.....56
5.5 本章小结.....60
第 6 章 总结与展望 .....61
6.1 论文工作总结.....61
6.2 未来研究方向.....61
参考文献.....63
致 谢.....65
作者简历及攻读学位期间发表的学术论文与研究成果 .....66

中国科学院计算技术研究所 鄢玺 2025-03-14 14:51:13

# 图目录

图 1.1 虚拟机运行概览图.....2
图 1.2 虚拟机动态编译流程图.....4
图 2.1 HotSpot 虚拟机内部视图.....7
图 2.2 类加载器的层次关系.....10
图 2.3 解释器传统指令分派.....15
图 2.4 解释器线索化指令分派.....16
图 2.5 解释器不使用栈顶缓存寄存器.....17
图 2.6 使用栈顶缓存寄存器.....17
图 2.7 Java 虚拟机解释与编译的关系.....18
图 2.8 方法调用计数器触发即时编译.....19
图 2.9 回边计数器触发 OSR 编译.....20
图 2.10 逆优化时的栈帧变化.....21
图 2.11 X86 和 MIPS 平台处理器性能对比.....26
图 2.12 X86 和 MIPS 平台虚拟机性能对比.....27
图 3.1 Java 虚拟机的分层编译.....29
图 3.2 X86 平台虚拟机分层编译性能提升.....30
图 3.3 C1 编译器基本结构.....31
图 3.4 C1 编译器代码调用路径.....32
图 3.5 LIR_Opr 类型继承关系图.....34
图 3.6 适配器示意图.....35
图 3.7 SPECjvm98 测试集下分层编译带来的启动性能提升.....40
图 3.8 SPECjvm98 测试集各测试项启动性能的提升.....40
图 4.1 解释器执行框架图.....45
图 4.2 解释器栈帧.....46
图 4.3 分派表结构.....46
图 4.4 字节码合成过程.....49
图 4.5 SPECjvm98 和 DaCapo 测试集下解释器平均性能提升.....50
图 4.6 SPECjvm98 和 DaCapo 测试项解释器性能.....51
图 5.1 Java 虚拟机参数分类.....54

图 5.2 OpenTuner 内建数据类型 .....55
图 5.3 SPECjvm98 和 DaCapo 测试项虚拟机调优测试.....57
图 5.4 SPECjvm98 和 DaCapo 测试集下虚拟机机平均性能提升.....58
图 5.5 SPECjvm98 和 DaCapo 测试集各测试项性能.....60

中国科学院计算技术研究所 鄢銮 2025-03-14 14:51:13

# 表目录

表 2.1 C1 编译器与 C2 编译器对比 ..... 13
表 2.2 模板解释器实现伪代码 ..... 14
表 2.3 Specjvm2008 的 MonteCarlo 测试项核心代码 ..... 20
表 2.4 分支预测产生的编译代码 ..... 23
表 2.5 SPECjvm98 各子测试项内容 ..... 24
表 2.6 DaCapo 各子测试项内容 ..... 25
表 2.7 X86 和 MIPS 平台处理器参数 ..... 25
表 3.1 虚拟机错误信息 ..... 37
表 3.2 finalize_oop_references 函数部分代码 ..... 37
表 3.3 iter 的重定位信息 ..... 38
表 3.4 出错位置处的 LIR 语句 ..... 38
表 3.5 const2reg 函数出错位置 ..... 39
表 3.6 mov_metadata 函数在 MIPS 平台上的实现 ..... 39
表 4.1 字节码执行‘1+1’ ..... 43
表 4.2 MIPS 汇编执行‘1+1’ ..... 44
表 4.3 栈顶缓存寄存器状态说明 ..... 47
表 4.4 isub 字节码对应的龙芯平台与 X86 平台汇编代码 ..... 48
表 4.5 部分高频字节码在测试项当中的占比 ..... 50
表 5.1 Java 虚拟机调优参数 ..... 57
表 5.2 SPECjvm98 和 DaCapo 各测试项最优启动参数 ..... 59

中国科学院计算技术研究所 鄢玺 2025-03-14 14:51:13

# 第1章 绪论

## 1.1 研究背景

自 90 年代初至今，Java[1]语言已经经过了 20 多年的发展，成为了人类计算机历史上应用广泛，影响深远的编程语言之一。Java 语言脱胎于 C++语言，吸收了 C++语言的一些优良特性，比如说继承和多态，同时，它也摒弃了 C++当中像多继承、指针这样晦涩难懂的概念。

Java 语言功能强大，简单易学，具有良好的平台无关性与安全性，方便通过网络传播。早在 1995 年，Java 语言就提出了“Write Once, Run Anywhere”的口号。如今，Java 语言的应用主要有三个方面，即 Java SE(Java Standard Edition)、Java ME(Java Micro Edition)和 Java EE(Java Enterprise Edition)，它们分别用于桌面端应用系统、移动设备与信息家电设备控制系统和企业级应用系统。其中，基于 Java SE 开发了非常知名的 IDE Eclipse 以及压力测试工具 Jmeter。当然，Java 技术中应用最为广泛的部分还是 Java EE，它主要用于 Web 框架，中间件的开发，并且涌现了很多优秀的开源软件，如 Spring、MyBatis、Tomcat 等，国内很多知名企业如阿里，京东等都是采用 Java EE 作为企业应用开发的解决方案。

Java 应用平台的核心是 Java 虚拟机，虚拟机为 Java 语言屏蔽了底层平台的硬件细节，提供了运行时环境，直接影响着上层应用的性能。随着 Java 应用场景的多样化发展，对于 Java 虚拟机的设计提出了严峻的挑战，当前 Java 虚拟机主要面临以下几个问题。

- 处理器兼容性问题。平台相关的 Java 虚拟机在 X86 等主流处理器架构上进行了多年的充分优化，但是，由于研发力度等原因，在龙芯处理器、申威处理器等国产处理器平台上[2]，存在一定程度上优化不足的问题。Java 虚拟机未能充分利用这些平台的处理器特性，导致性能较主流处理器平台有一定的差距，一定程序上限制了国产处理器的市场化推进工作。
- 启动性能问题。在移动端，Java 虚拟机也为 Android 平台上丰富的手机应用提供了运行时环境。此类客户端型 APP 应用图形界面较多，与用户

的交互频繁，启动性能与响应速度会对用户体验产生直接影响，决定产品竞争力。传统的 Java 虚拟机性能优化技术多针对服务端高性能应用，忽略了对虚拟机快速响应能力的提升。

- 应用场景适配问题。Java 虚拟机启动参数种类繁多、功能各异。同样的应用以不同参数配置启动，在性能上可能会有很大的差别。面对不同的应用场景，选择合适的启动参数，最大化发挥虚拟机性能，对于提升生产效率意义重大。

综上所述，对 Java 虚拟机性能优化技术展开研究，无论是在理论上还是在实际应用当中，都具有重大意义。

## 1.2 Java 虚拟机简介

截止到 2018 年 1 月，TIOBE 编程语言排行榜[4]上，Java 依旧占据着榜首。Java 程序设计语言是一门纯粹的面向对象的设计语言，它拥有抽象、封装、继承和多态这样的面向对象编程特性。不同于 C/C++这类的静态编译型语言，Java 源代码会先经过 Javac 编译器转换为字节码[3]文件，再交由 Java 虚拟机执行。如图 1.1 所示。

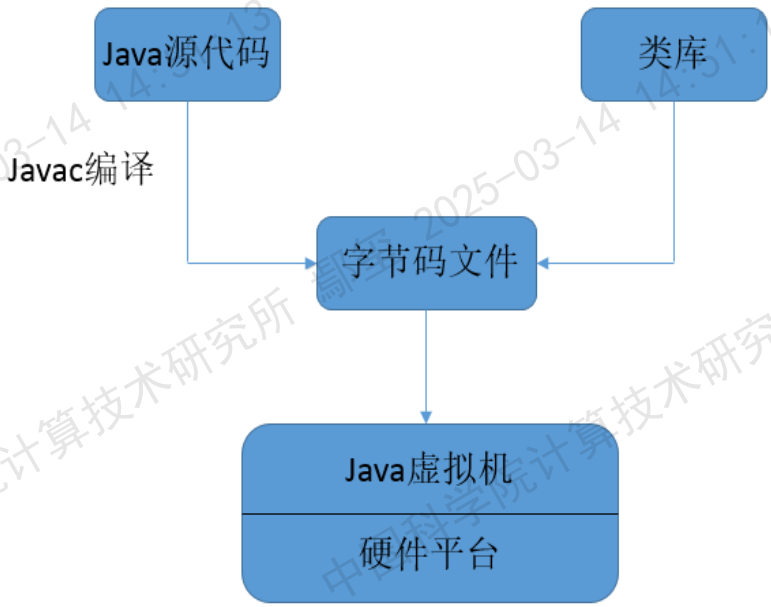


**图 1.1 虚拟机运行概览图**

**Figure 1.1 Virtual Machine running overview**

Java 虚拟机为 Java 语言提供了运行时环境，它负责读入字节码文件，进行模拟执行，是影响 Java 程序性能的关键因素。Java 虚拟机是 Java 实现跨平台、安全可靠和其他许多高级语言特性的基础。为了支持上述这些特性，Java 虚拟机在设计复杂的同时也面临着许多性能问题。首先，为了实现跨平台的特性，Java 源程序以字节码这种中间表式形式通过网络传播。在执行 Java 程序时，Java 虚拟机需要先将字节码转换为平台相关的二进制指令，再将这些转换后的二进制代码交由处理器直接执行，上述转换是在 Java 程序运行时所进行的操作，是影响 Java 程序性能的主要因素。其次，为了避免像 C/C++语言当中出现的不安全操作引发系统崩溃，Java 虚拟机内置了多种安全性检查机制，如数组越界检查、空指针检查和字节码文件完整性检查等。这些额外的安全性检查操作占用了大量的 Java 程序运行时时间，进一步限制了 Java 程序的性能。最后，Java 虚拟机所支持的自动内存管理（垃圾收集）也增加了运行时开销。

为了解决上述问题所带来的挑战，尽可能地提高 Java 程序性能，现代 Java 虚拟机系统的核心组件执行引擎都需要经过精心设计。执行引擎主要负责字节码向机器码的转换。Java 虚拟机的执行引擎包括解释器(interpreter)[5]和即时编译器(just-intime compiler, JIT)[6]这两部分。解释器的模块简单，利于实现和维护，所以早期 Java 虚拟机的执行引擎只有解释器这一个组件。但解释器没有对方法翻译过程进行优化，运行效率过于低下，为人们所诟病。为了解决解释执行性能过于低下的问题，人们加入了 JIT 编译器来提高虚拟机性能。因为 JIT 编译器是在程序运行期间对方法或循环等代码块进行编译执行的，所以这种编译方式又被称为动态编译[7]。根据相关研究，JIT 编译器所生成的本地机器代码的执行效率是解释执行的 10 倍以上[8]。Java 虚拟机在实际运行中会采用解释器和编译器相结合的方式，图 1.2 展示了 HotSpot 虚拟机中的动态编译过程。HotSpot 虚拟机是 Sun 公司研发的遵循 Java 虚拟机规范[3]的一种 Java 虚拟机实现，这也是现在应用最为广泛的 Java 虚拟机了，Sun 于 2006 年开源了所有 HotSpot 虚拟机代码，现在由 OpenJDK 开源社区[9]管理。

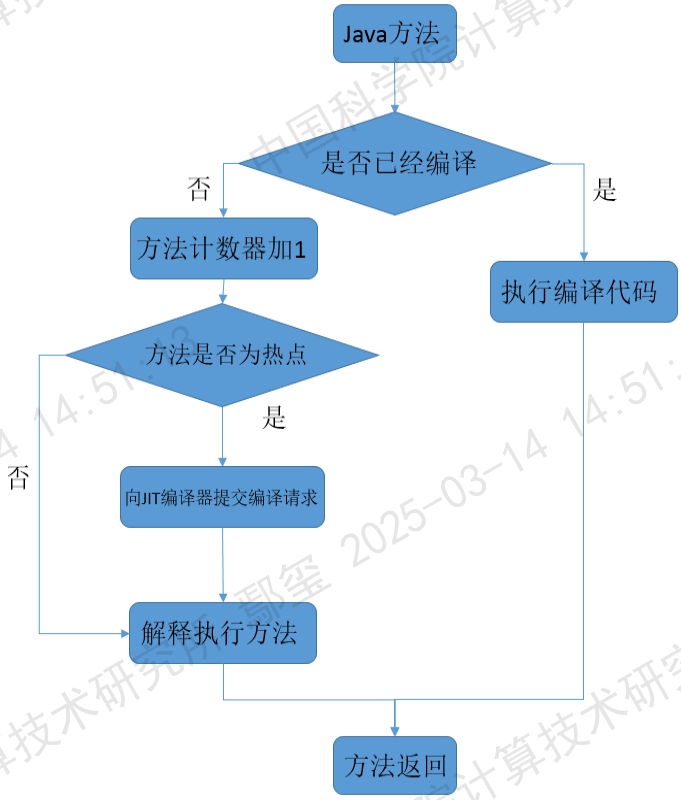


**图 1.2 虚拟机动态编译流程图**

**Figure 1.2 Virtual Machine dynamic compilation flowchart**

由图 1.2 可以看到，实际运行时 Java 虚拟机会对方法进行热点识别，对执行较为频繁的方法或循环进行编译执行，而一般的方法就由解释器来运行。这是因为动态编译占用了 Java 程序的运行时时间，如果所有方法都编译执行的话，效率可能反而会降低。为此，虚拟机为每个方法都维护了一个计数器，如果方法调用次数达到一定的阈值，那么这样的方法就被当作热点方法，对热点方法进行编译往往能获得较好的执行性能。

同时，现代 Java 虚拟机为了能够充分利用多核处理器的性能，大都采用了多线程的编译技术来实施 JIT 编译，这种后台编译技术使得编译线程与应用线程同时运行，最大程度上避免了编译对程序执行的干扰。

## 1.3 本文的主要内容

本文对于 Java 虚拟机的优化研究，主要工作内容有以下三个方面。

1. 分层编译的实现

Java 虚拟机存在平台兼容性问题，当前的虚拟机是针对 X86 处理器架构进

行设计的，在 MIPS、PowerPC 等架构[10]上，实现不够完善或是优化力度不足，导致相应平台的虚拟机不能很好地利用处理器的特性来充分发挥性能。针对兼容性问题，本文实现了 MIPS 平台上 Java 虚拟机的分层编译功能，使得虚拟机在 MIPS 平台上的启动性能得到提升。

2. 解释器性能优化

解释器是 Java 虚拟机执行引擎的重要组成部分，它决定了应用程序的启动速度。针对当前 Java 虚拟机启动性能不足的问题，首先对虚拟机解释器的运行原理进行说明，再通过对字节码的量化分析指出解释器的性能瓶颈，最后有针对性地提出一种合并字节码的优化方法。该方法可以有效减少运行时字节码数量，进而减少跳转指令数目，达到优化效果。

3. 虚拟机参数调优

Java 虚拟机的调优工作一直是非常热点的话题，不同的启动参数，可能对程序性能影响很大，针对某个特定的应用场景，找到最优的参数组合是调优的目的。本文在 MIPS 平台的 Java 虚拟机基础上，选取部分启动参数，利用 OpenTuner 自动调优框架进行调优，提升虚拟机在特定类型应用中的性能。

本文的实验主要是在龙芯最新产品 3A3000 处理器[11]上进行的。龙芯系列处理器芯片是龙芯中科技术有限公司[12]自主研发的基于 MIPS 架构的处理器芯片。分层编译的实现使得虚拟机的启动性能有了 33%的提升，解释器优化使得解释器的性能提升在 4.7%，虚拟机参数调优带来的性能提升最高为 86%。相关的代码已经提交到龙芯 OpenJDK8[9]代码库当中。除了分层编译的实现以外，解释器性能优化技术与虚拟机参数调优方法对于其他平台也同样适用。

## 1.4 本文的组织结构

本文一共分为 6 章，内容分别如下。

第 1 章绪论部分，首先介绍了研究的背景与意义，再对 Java 虚拟机的相关原理进行简单的说明。阐明了 Java 虚拟机在 Java 程序执行过程当中所起到的重要作用以及 Java 虚拟机的性能优化工作的理论意义与实际应用价值。

第 2 章 Java 虚拟机背景介绍，先对 HotSpot 虚拟机的主要组成部件进行了说明，再重点介绍了虚拟机解释器与编译器常用的优化方法，最后介绍了实验用到

的评估虚拟机性能的标准测试集程序，并通过与 X86 平台的性能对比说明 MIPS 平台虚拟机优化的必要性。

第 3 章分层编译的实现，介绍了实现 MIPS 平台虚拟机 C1 编译器的实现过程，先对 C1 编译器的原理进行介绍，说明 C1 编译器的实现弥补了当前 MIPS 平台 Java 虚拟机只有解释器与 C2 编译器的短板，使得 MIPS 平台可以支持分层编译，再以一个典型的调试样例说明 MIPS 平台上分层编译实现过程当中遇到的问题以及解决思路。

第 4 章解释器性能优化，主要是关于 Java 虚拟机的解释执行引擎，首先对 Java 解释器的工作原理进行详细地说明，分析解释器当中存在的性能瓶颈，再有针对性地提出一种合并字节码的优化方法，减少运行时字节码数量，进而减少跳转指令数量，达到优化效果。

第 5 章虚拟机参数调优，先介绍了程序自动调优的优势以及对虚拟机参数调优的必要性，再介绍了利用 OpenTuner 这个自动调优框架对 MIPS 平台上的 Java 虚拟机进行调优的过程以及优化效果。

第 6 章总结与展望，对前 5 章的内容以及在研究过程当中的收获与体会进行了总结，分析可能存在的问题，进而指出下一步的研究方向。

# 第2章 Java 虚拟机背景介绍

本章首先对 Java 虚拟机的基本概念、组成结构与功能进行说明。其中，重点介绍了虚拟机的运行时数据区以及类加载器、垃圾收集器[13]、执行引擎和本地方法接口这几大核心部件。再列出了 Java 虚拟机当中的经典优化方法，主要展示了解释执行与动态编译中常见的优化手段。最后，介绍了虚拟机性能评测的标准测试集并对比了 X86 和 MIPS 平台虚拟机的性能。

## 2.1 Java 虚拟机基础

HotSpot 虚拟机的核心部件如图 2.1 所示，主要有类加载器、执行引擎、垃圾收集器以及运行时数据区等。各个部件的功能如下。

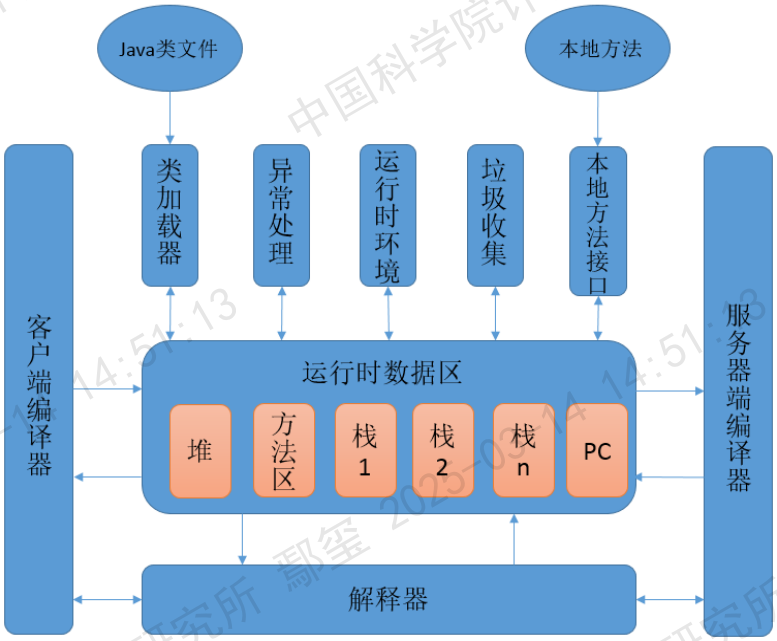


**图 2.1 HotSpot 虚拟机内部视图**

**Figure 2.1 HotSpot Virtual Machine internal view**

1. 运行时数据区：由堆、方法区、栈、PC 等组成，存放虚拟机运行时所需要本地代码，类文件等重要数据结构，支持方法的解释或编译执行。

2. 类加载器：负责从本地文件或是网络当中读取字节码文件到虚拟机当中，

并生成虚拟机相关的数据，加载过程中还会对字节码文件进行校验，确保安全性与正确性。

3. 执行引擎：对 Java 程序解释或是编译执行。

4. 垃圾收集器：负责分配与回收内存，管理虚拟机的运行时数据区。

5. 本地方法接口[14]（JNI，Java Native Interface）：为虚拟机与本地方法的交互提供了统一的接口。

了解 Java 虚拟机的核心部件，加深对虚拟机的理解，有利于下文性能优化工作的展开。下面各节分别介绍这些模块。

### 2.1.1 运行时数据区

运行时数据区操作系统分配给 Java 虚拟机，并由虚拟机自行管理的内存区域，这些区域有不同的用途，以及创建与销毁时间，根据《Java 虚拟机规范》[3]的规定，如图 2.1 所示，虚拟机会把运行时数据区具体分为程序计数器、虚拟机栈、本地方法栈、堆这 4 个部分，下面介绍各区域作用。

- 程序计数器。这是一块较小的内存空间，只在虚拟机解释执行时有效，它始终指向当前要执行的字节码的位置，解释器在运行的时候，通过改变这个计数器的值，不断读取下一条字节码。如果正在执行的是本地方法，那么这个计数器的值为空。每个 Java 线程都拥有各自的程序计数器。
- 方法区。方法区是 Java 线程共享的内存区域，用来存储从字节码文件加载而来的类元数据、常量、静态变量以及 JIT 编译器生成的本地方法代码等。Java 虚拟机的规范在逻辑上把方法区归为堆的一部分，但是虚拟机厂商可以根据需要灵活实现。方法区除了和堆一样不需要连续的内存空间，还可以选择不实现垃圾回收。因为这个区域存储的是类相关的信息，垃圾收集行为是比较少见的。
- 堆。在大多数情况下，Java 堆是虚拟机当中最大的一块内存区域了，它为所有 Java 线程共享，随着虚拟机的启动而创建。堆的作用是用来存储 Java 对象的实例数据，它也是垃圾收集器的主要工作区域，根据虚拟机规范的描述，Java 堆只要在逻辑上空间连续即可，物理上可以不连续。目前主流虚拟机的 Java 堆部分都是可扩展的（HotSpot 当中通过-Xmx

和-Xms 参数控制），当堆的空间无法满足对象分配需求时，会抛出 OutOfMemoryError 异常。

- 栈。栈同程序计数器一样是线程私有的内存区域。它的生命周期与 Java 线程相同。虚拟机栈作用与 C 语言执行时用到的栈类似，它描述了虚拟机内方法执行时的情况：虚拟机会为每个执行方法创建一个栈帧，栈帧当中存储了局部变量表、动态链接、操作数栈、方法入口等信息，方法从调用到返回的过程，就对应着一个栈帧从入栈到出栈的过程。在虚拟机规范中，栈被进一步分为 Java 栈和本地方法栈，分别支持 Java 方法和 C 语言方法的执行，不过在具体实现时，可以灵活选择，比如 HotSpot 虚拟机就把本地方法栈和 Java 栈合二为一。

### 2.1.2 类加载器

Java 源代码当中存储着程序的执行逻辑，经过 Javac 编译器被编译为字节码文件。当虚拟机需要用到某个 Java 类时，就会通过类加载器读取其对应的字节码文件到内存中，构造出可以被虚拟机直接使用的数据结构。可以看到，Java 程序在执行当中根据需要来加载相应模块，具备很好的动态扩展性。同时，类加载器不限于从本地加载字节码文件，也可以从网络获得相关程序，赋予了 Java 很好的网络移动性。类加载器最初是为了 Java Applet 技术的需求而开发的，虽然现在 Java Applet 已经被 HTML 技术取代，但是类加载器却广泛应用在 OSGi、代码加密、类层次划分、热部署等领域，成为 Java 技术当中的关键一环。

从 Java 程序员的角度来看，类加载器可以分为以下四类。

- 启动类加载器。这个类加载器由 C++语言实现，是虚拟机的组成部分，它在虚拟机启动时创建，负责加载<JAVA_HOME>/lib 目录中 Java 核心类的字节码文件，无法被 Java 开发人员直接使用。
- 扩展类加载器。这个加载器的实现在 sun.misc.Launcher$ExtClassLoader 类当中，它主要负责<JAVA_HOME>/lib/ext 目录下的类的加载，也可以通过 java.ext.dirs 系统变量来指定加载路径，开发者可以直接使用扩展类加载器。
- 应用程序类加载器。这个类加载器实现在 AppClassLoader 类中，又被称

为系统类加载器，它负责加载用户 classpath 路径中的所有类，可以被开发者直接使用，一般应用程序的类都是由这个类加载器加载的。

- 用户自定义类加载器。用户可以通过继承 ClassLoader 类并实现其 findClass 方法来定义自己的类加载器。自定义类加载器一般有两种应用场景，一种是对字节码文件进行加密，防止被第三方读取并反编译，另一种是从非标准来源读取代码，比如说从数据库或是云端。

应用程序是由这四类加载器配合进行加载的，这些加载器遵循如图 2.2 所示的层次关系。这种层次关系又被称为双亲委派模型。

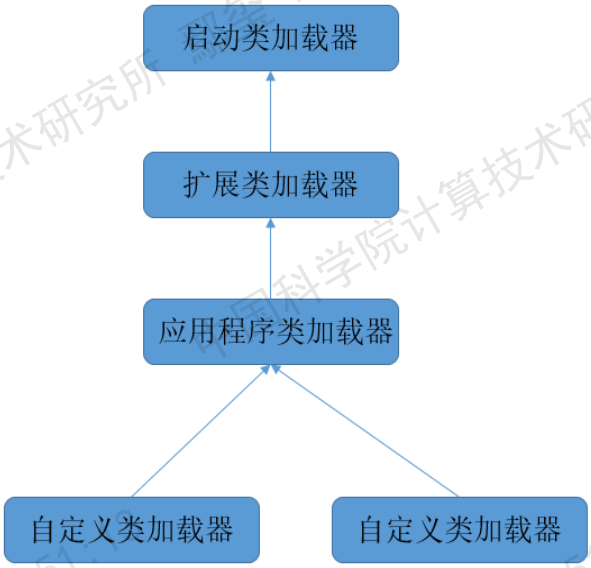


**图 2.2　类加载器的层次关系**

**Figure 2.2　ClassLoader hierarchy**

双亲委派模型的工作过程是：如果一个类加载器收到加载类的请求，它首先不会自己立即去加载，而是把这个请求向上传递到它的父类加载器，父类加载器也遵循同样的逻辑直到请求到达启动类加载器当中，只有当父类加载器给出反馈，表明无法加载该类时，子加载器才会尝试去加载。

通过双亲委派模型组织起来的类加载关系，使得 Java 类随着它的加载器一起有了优化级关系。比如 Java 的核心类 java.lang.Object，所有 Java 类默认继承此类，它位于 rt.jar 当中，无论是哪一个等级的加载器要加载它，最终请求都是由启动类加载器来执行，这样就保证了被加载进虚拟机当中的 Object 类的惟一

性，即使用户自定义了 Object 类，也不至于被加载到虚拟机当中，破坏 Java 类的继承关系。

### 2.1.3 垃圾收集器

相比于 C/C++这样的传统编程语言，Java 有特色的就是自动内存管理功能了，这个功能依靠 Java 虚拟机当中的垃圾收集器部件实现，它的出现，使得程序员不用再关心内存的释放问题，垃圾收集器会自动识别出内存中不再被程序使用的部分，并释放这些空间以供下次使用。正是有垃圾收集器，Java 程序不会像 C/C++程序那样，容易引发内存泄漏的问题。当然，垃圾收集过程中会使得 Java 程序的短暂停止，影响执行效率。因此设计新的垃圾收集器，满足管理大的内存空间同时又有很短的暂停时间是一个很大的挑战。

垃圾收集（Garbage Collection，GC），即自动识别并回收内存当中不再被使用的区域，垃圾收集需要解决两个问题：1）如何对所有不再被使用到的对象进行标记；2）怎样清理垃圾对象。

第一个问题有以下两种算法[15]可以解决。

引用计数算法。引用计数算法为每一个对象都维护着一个计数器，每当一个新的地方引用这个对象，计数器数目就会增加 1；当引用失效时，计数器数值减 1。这样，当计数器的数目为 0 时，就说明该对象不再被使用，可以进行回收了。虽然工作机制简单，但是这个算法还存在着明显的问题，那就是无法解决循环引用。比如 A 对象引用 B 对象，同时 B 对象也引用着 A 对象，但是这两个对象都不再被使用。面对这样的场景，引用计数算法就无法回收内存。

可达性分析算法。现代 Java 虚拟机的垃圾回收器都是使用这个算法来识别无用对象的。算法的基本思路是先确定一个根对象的集合作为 GC 根，Java 语言当中虚拟机栈、方法区中的静态变量、方法区中常量和本地方法这些位置的对象都可以作为根结点，从这些根结点出发搜索对象引用路径，路径上的对象都是存活对象。通过可达性分析算法，所有的存活对象都被成功标记。

第二个问题有以下三种算法[15]可以解决。

标记-清除算法。这种算法分为标记和清除两个阶段，第一阶段标记用的是上面介绍的引用计数或者可达性分析算法，第二阶段回收所有垃圾对象。这种算

法有两个弊端，一个是标记与清除阶段的效率都不高，另一个是回收过后内存空间碎片过多，导致无法继续分配大的内存。不过其他垃圾收集算法都是在此算法基础之上作出改进而提出来的。

复制算法。为了解决垃圾回收效率问题，复制算法被提出来了。复制算法将内存空间划分成大小相等的两块，每次系统只在一块上分配对象，进行垃圾回收的时候，直接将一块中的存活对象复制到另一块当中，然后在系统在另一块上分配对象，如此循环往复。可以看到，这种算法的垃圾回收效率很高，不过缺点是浪费了一半的内存空间，代价过高。

标记-整理算法。标记整理算法的标记阶段同标记-清除算法，不过在进行垃圾回收的时候，将所有存活对象都向内存的某一端移动，然后清除掉存活对象以外的空间。这种算法不会产生内存碎片，不过垃圾收集的效率不高。

### 2.1.4 执行引擎

Java 虚拟机当中的执行引擎分为解释器与即时编译器这两类。它们的交互关系如图 1.2 所示。

解释器是 Java 虚拟机当中最早的也是最基础的部件了。1996 年 1 月 23 日，Sun 发布的 JDK 1.0 版本当中，Java 语言首次拥有正式的运行环境，这版 JDK 当中所带的 Classic VM 虚拟机(Hotspot 虚拟机的始祖)就只有解释器模块。解释器就是对 Class 文件当中的字节码依次读入并解释执行的部件。由于每次都要对特定字节码做出翻译，再将其对应的机器代码交由 CPU 执行，所以解释器执行 Java 程序的时候比较耗时，效率低下，不能满足服务器端对应用程序的性能要求。早期 Java 为人们诟病最多的也正是低下的执行性能，即便拥有跨平台这些优异的特性，依旧无法大规模投入商用市场。

Java 虚拟机当中的即时编译器又分为 C1 编译器[16]（client compiler，客户端编译器）和 C2 编译器[8]（server compiler，服务端编译器）两种。

C2 编译器。为了尽可能地提升 Java 虚拟机的性能，引入了动态编译的方法，不同于 C 语言这样的静态编译语言，Java 程序并不是在开始运行之前就被编译成本地机器代码，只是由 Javac 这个前端编译器编译成为字节码这样的中间表示形态。在开始运行之后，由 Java 虚拟机内部的编译器决定何时编译，将 Java 字

节码转化为本地机器码，C2 编译器就是 Java 虚拟机的动态编译器之一，它的引入就是要解决 Java 程序性能低下的问题，利用程序运行时所收集的信息，识别热点函数，有针对性的进行编译。C2 编译器的出现，大大提高了 Java 程序的性能。相比于静态编译，程序运行时可以收集到更多的优化信息，优化程度要高，随着 C2 编译器的不断完善，如今的 Java 虚拟机的即时编译器已经可以编译出执行效率很高的本地代码了，某些方面甚至已经超过了 GCC 这样的传统编译器。

C1 编译器。C1 编译器是 Java 虚拟机中另一个即时编译器了，它的出现是为了弥补 C2 编译器的一个短板。即时编译器其实有一个难以平衡的问题，那就是编译时间与优化程度的选择。如果要生成优化程度高的代码，编译时间就比较长，而即时编译器是在运行时编译的，编译时间长虽然可以充分优化，程序峰值性能高，但是对原程序的启动时间影响很大。在服务器上运行的程序可以使用 C2 编译器，因为这种程序运行时间长，要求峰值性能高。对于客户端程序来说，情况就不一样了，很可能编译还没有完成时程序就运行结束了，所以这类程序要求编译器编译速度快。C1 编译器正是由此而诞生的。表 2.1 是 C1 编译器与 C2 编译器的一个简单对比。

**表 2.1 C1 编译器与 C2 编译器对比**

**Table 2.1 Comparison between C1 compiler and C2 compiler**

| 即时编译器 | 编译时间 | 生成代码质量 | 运行效率 |
|---|---|---|---|
| C1 编译器 | 短 | 低 | 低 |
| C2 编译器 | 长 | 高 | 高 |

### 2.1.5 本地方法接口

Java 作为一门纯面向对象的高级语言，抽象、继承、封装和多态这样的语言特性给编程带来了很大的便利。不过，它也有一些本身的缺点。比如执行性能不如 C/C++，无法操作系统底层。本地方法接口（JNI，Java Native Interface）就提供了一个标准，使得 Java 语言通过虚拟机可以与 C/C++代码，甚至是汇编语言交互。本地方法接口有以下的优点。

- 实现简单。JNI 没有对 Java 虚拟机的实现添加任何限制，各虚拟机厂商

可以很方便地在虚拟机中加入对 JNI 的支持。

- 提高应用程序性能。Java 虚拟机帮助 Java 语言实现了底层的功能，但是同时相比于 C/C++语言来说效率不高，通过 JNI 的方式，Java 可以直接调用 C 来实现相关功能，性能会有所提升。
- 对底层功能的支持。JNI 的设计目标之一就是让 Java 语言可以通过调用 C 语言这些底层语言，从而支持很多平台相关的特性，如文件 I/O、网络连接、创建线程等。

## 2.2 Java 虚拟机常见优化方法

执行引擎是 Java 虚拟机的核心部件，直接决定 Java 虚拟机性能表现。本节对 Java 虚拟机解释器与即时编译器的常见优化方法进行介绍。

### 2.2.1 解释器优化技术

1. 线索化指令分派[17]

解释执行的基本单位是字节码。解释器的工作方式实际上就是按字节码读入字节码文件，并按照取指、指令分派、取操作数和执行这四个步骤对每个字节码依序解释执行。这四个步骤的含义如下。

- 取指令：获取下一个字节码的位置。
- 指令分派：根据当前字节码的操作码，跳转到相应例程执行。
- 取操作数：从栈上获取字节码指令所需操作数。
- 执行：执行该字节码的实际汇编代码。

最容易想到的实现方式如表 2.2 这段伪代码所示。

**表 2.2 模板解释器实现伪代码**

**Table 2.2 Pseudo-Code of template interpreter**

```
While(next_bytecode) {
    bytecode = getNextBytecode(); //获取下一条字节码
    switch(bytecode) {
        case nop : … break;
        case iload : … break;
        ….
```

这段伪代码所做的工作很简单，就是不断获取下一条字节码，再根据该字节码内容执行相应动作。可以看到，这种解释器的实现方式指令分派都集中在了 while 循环当中，如图 2.3 所示（实线表示间接跳转，虚线表示直接跳转）。

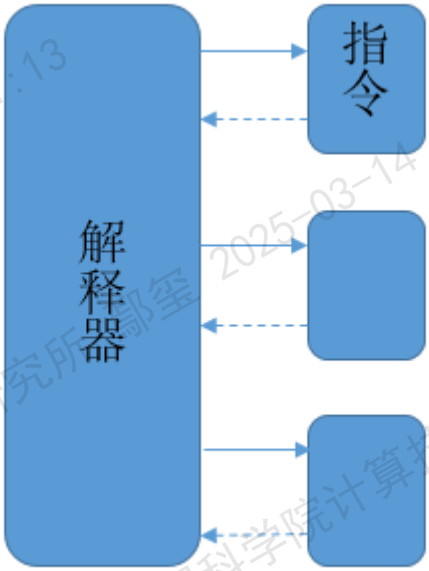


**图 2.3 解释器传统指令分派**

**Figure 2.3 Interpreter traditional instruction dispatch**

虽然这种指令分配方式编码容易，但是非常低效，每个字节码的执行都需要一次直接跳转与间接跳转。

指令分派是完成的功能是当前字节码执行完成后，继续找到下一条要执行的字节码。解释执行的性能瓶颈一般就在指令分派上。虚拟机当中大多数的字节码，如 imul、idiv、i2c 这类的，都只完成非常简单的功能。如乘除法操作、类型转换这些，通常 1~2 条汇编指令即可实现上述操作。而指令分派通常要十多条汇编代码才能完成，功能较为复杂。此外，指令分派中至少含有一条执行开销较大的间接转移类指令。因此，指令分派在程序解释执行期间占有较大比例的执行时间。

在 Hotspot 虚拟机当中，解释器使用了线索化指令分派[17]的方式来优化效率，它将指令分派的工作合成到了每一条字节码指令的后面。指令实际的执行流程图如图 2.4 所示。

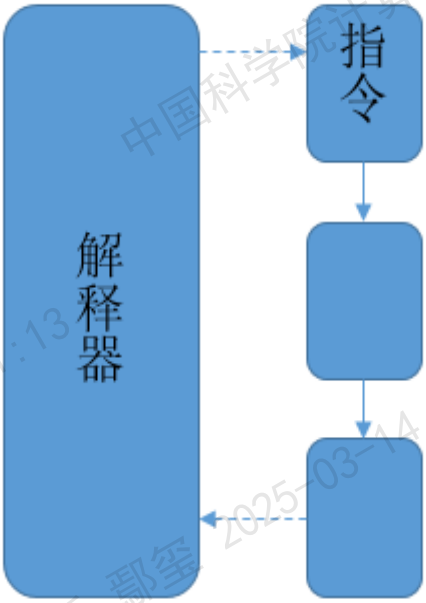


**图 2.4 解释器线索化指令分派**

**Figure 2.4 Threaded Code of interpreter**

可以看到，采用了线索化指令分派后的解释器，相比于 Switch 方式的实现，减去了字节码之间的直接跳转指令部分，大大提高了执行效率。

2. 栈顶缓存寄存器

Java 虚拟机的解释执行是基于操作数栈这种数据结构[5]的，字节码需要把操作数从局部变量表部分转移到栈上才能继续操作。这种基于栈式的操作虽然指令设计方便，但是效率低下，因为对于栈的操作都是访存。相关研究表明，解释执行过程中，访存指令占有相当大的比例[18]。相对于寄存器来说，内存访问始终是 CPU 执行速度的瓶颈。为了尽可能减少内存访问的次数，Java 虚拟机当中引入了一种被称栈顶缓存的优化技术，这种优化技术的关键就是使用了一个通用寄存器专门用来保存栈顶数据。以两条 iadd 字节码为例说明，优化之前，访存的次数如图 2.5 所示。

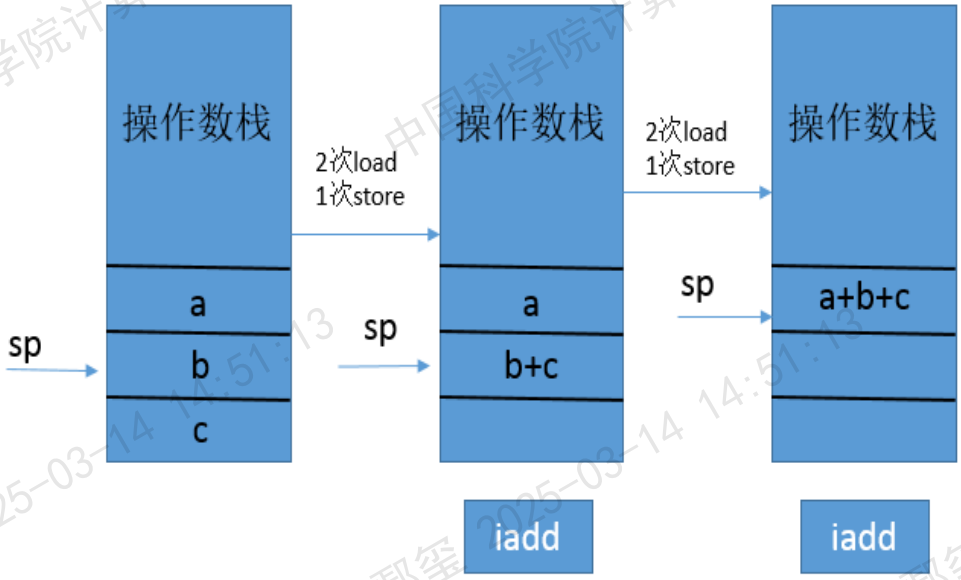


**图 2.5 解释器不使用栈顶缓存寄存器**

**Figure 2.5 Interpreter without stack caching**

引入了栈顶缓存寄存器后，访存次数如图 2.6 所示。

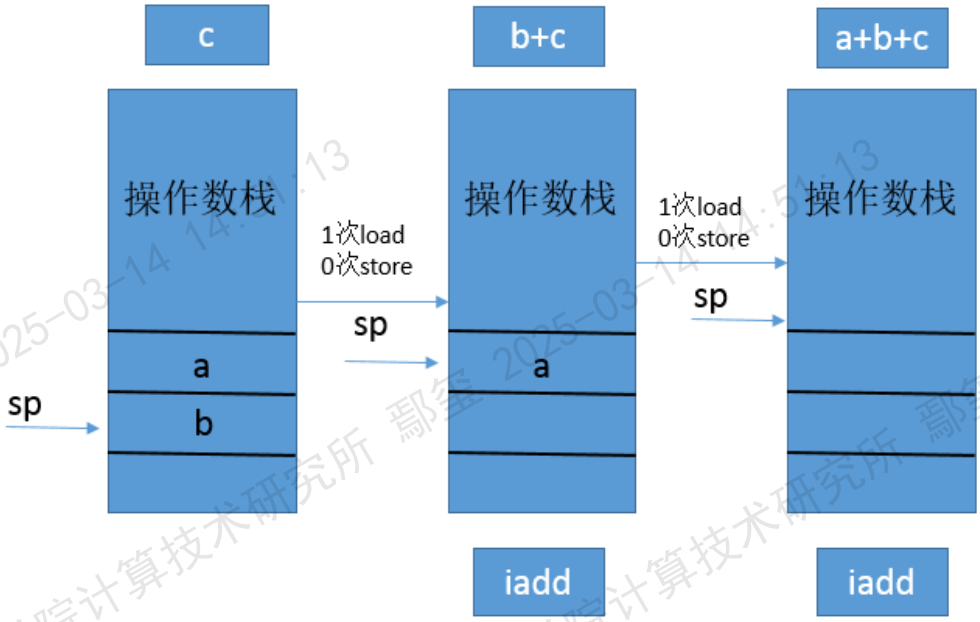


**图 2.6 使用栈顶缓存寄存器**

**Figure 2.6 Interpreter with stack caching**

在图 2.5 中，有 6 次访存操作，经过优化后，在图 2.6 中，可以看到，访存

操作的次数下降为了 2 次，大大提高了解释器的效率。

在 Java 虚拟机当中，栈顶缓存寄存器作为一种重要的优化手段，直接影响了分派分派、字节码的本地代码这些解释器关键部分的实现。不了解它的话，会对 Java 虚拟机源代码的理解造成不小的困难。

### 2.2.2 编译器优化技术

1. 热点探测技术[19]

Java 虚拟机可以指定以解释执行的方式运行 Java 程序，但更多的时候，Java 程序是以混合执行的方式在 Java 虚拟机内部运行。如图 2.7 所示。

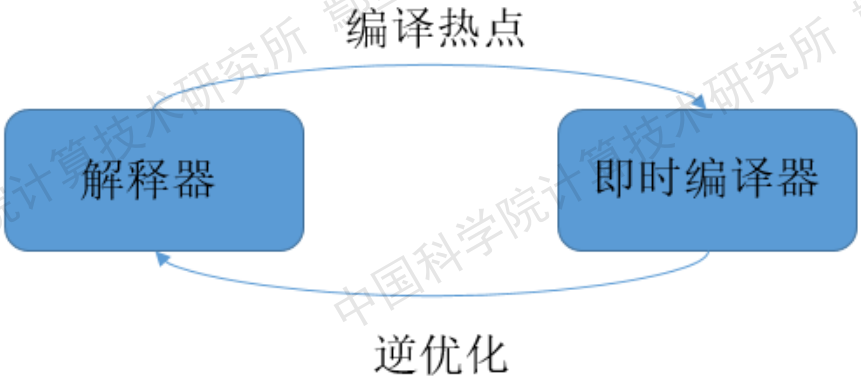


**图 2.7 Java 虚拟机解释与编译的关系**

**Figure 2.7 The relationship between interpretation and compilation in Java Virtual Machine**

可以看到，Java 程序经过 Javac 编译生成的字节码文件，要么是经过解释器解释执行，要么是通过 JIT 即时编译生成机器代码执行。那么什么时候解释，什么时候编译呢？

这时候就用到了热点探测技术（频繁调用的函数就称为热点），主流的热点探测技术有以下两种。

- 基于采样的热点探测技术，这种方法定期检查方法调用栈的顶部，如果一个方法经常出现的话，说明此方法比较活跃，可以进行编译。这种探测技术实现起来简单、高效，还可以对方法调用链有清晰的了解，但是准确度不高，容易受到像方法阻塞等因素的影响。

- 基于计数器的热点探测技术，这种热点探测技术为每个方法都单独维护计数器，用于在运行时统计方法执行的次数，当某一方法被执行的次数超过启动时就设置好的阈值时，就被识别成为热点方法，Java 虚拟机就会启动 JIT（Just-In-Time）编译器对该方法进行编译。这种探测技术识别热点方法较为精准，不容易受到其他因素的影响，但是实现难度不低。

其实在 HotSpot（OpenJDK 的 Java 虚拟机实现）内部采用的是基于计数器的热点探测技术，每个方法都会有两个计数器，分别是方法调用计数器（Invocation Counter）与回边计数器（Back Edge Counter），方法调用计数器用来统计方法被调用的次数而回边计数器统计方法内循环的执行次数，这两种计数器有各自的阈值，比如方法调用计数器在 OpenJDK8 的 Hotspot 虚拟机当中，C1 编译器和 C2 编译器的阈值分别是 1500 和 10000，这个值通过启动时设置 CompileThreshold 参数来改变。

这两种计数器分别应用于不同的编译场景，方法调用计数器的编译交互过程如图 2.8 所示。

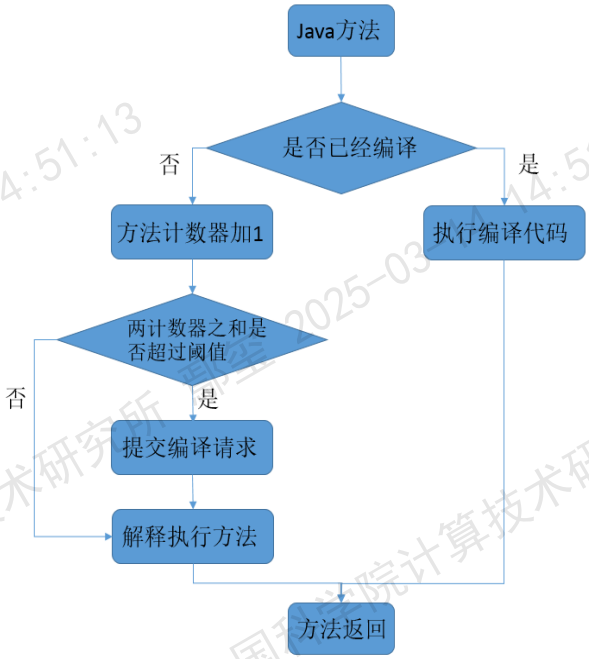


**图 2.8 方法调用计数器触发即时编译**

**Figure 2.8 Dynamic compilation invoked by Invocation Counter**

如果只用方法计数器的话，可能会忽略掉某些热点代码，比如该方法的调用

次数较少，但是方法内部的循环次数执行较多。一个很典型的例子就是 SPECjvm2008[20]（Java 虚拟机性能测试集）当中的 MonteCarlo 测试项，该测试项的核心代码如表 2.3 所示。

**表 2.3　Specjvm2008 的 MonteCarlo 测试项核心代码**

**Table 2.3　Core code of MonteCarlo in SPECjvm2008**

```
public final double integrate(int numSamples) {
    …
    for (int count = 0; count < 16777216; count++) {
        …
    }
}
```

从代码中可以看到，integrate 内部 for 循环的循环次数达到 16777216 次之多，如果不对循环进行计数，只对该方法进行解释执行，会严重影响虚拟机性能。

循环回边计数器与编译器交互过程如图 2.9 所示。

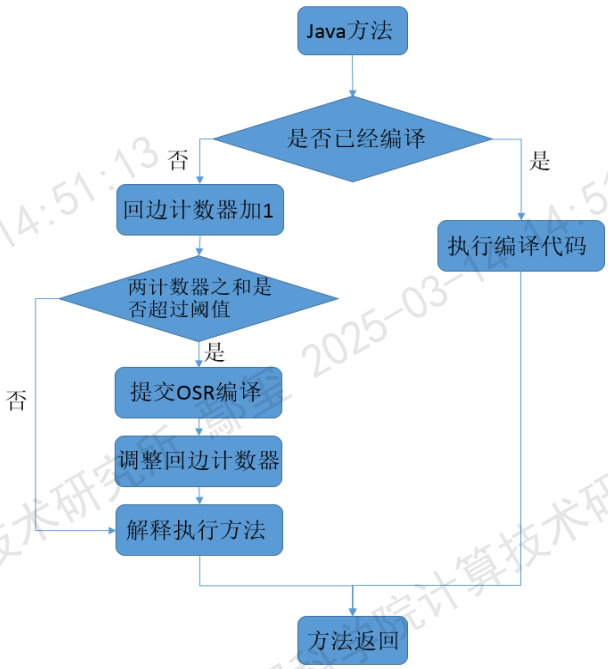


**图 2.9　回边计数器触发 OSR 编译**

**Figure 2.9　OSR compilation invoked by Back Edge Counter**

循环回边计数器触发的是被称为 OSR[21]（On-Stack Replacement）的编译类

型。下面对 OSR 这种虚拟机当中常用的技术做一个简单的介绍。

OSR 是一种运行时动态替换函数栈帧的技术，被用来在执行引擎不同优化层级之间切换，比如上面说的循环回边计数器触发编译后，当编译完成时，之前的解释方法栈帧可能还在栈顶，这时就需要 OSR 技术将解释方法栈帧替换为编译方法栈帧了。Java 虚拟机当中，OSR 技术常用的场景有以下几种。

解释方法栈帧替换为编译方法栈帧，典型的就是循环回边计数器触发编译后发生的，这种情况下采用 OSR 技术避免了在一个方法循环当中解释执行时间过长，对于提升虚拟机的测试集得分非常有效。

编译后的方法执行过程当中发生逆优化（虚拟机当中称为 deoptimization）而要进行栈帧替换。在虚拟机的 C2 即时编译器进行编译时，为了尽可能地提高生成代码质量，常常会做出非常激进的优化，比如对于不常调用的 if 分支，编译器可能不会对这部分代码进行编译，而只留下一个 uncommon trap 的标记，实际执行到这个分支时，再进行逆优化，即采用 OSR 技术把方法切换到解释执行模式，解释器作为最稳定、不容易出错的执行方式，充当了 C2 编译器激进优化时的逃生门。图 2.7 中展示的从编译器到解释器的路径就是逆优化的体现。

方便方法的调试[22]。在编译器进行编译的时候，会使用到像方法内联（inlining）、全局代码外提（global code motion）等优化技术，生成代码与源代码的差别很大，不易进行调试，为了方便调试的进行，就会使用到逆优化了，把原来经过内联的编译栈帧还原为解释方法栈帧。如图 2.10 所示。

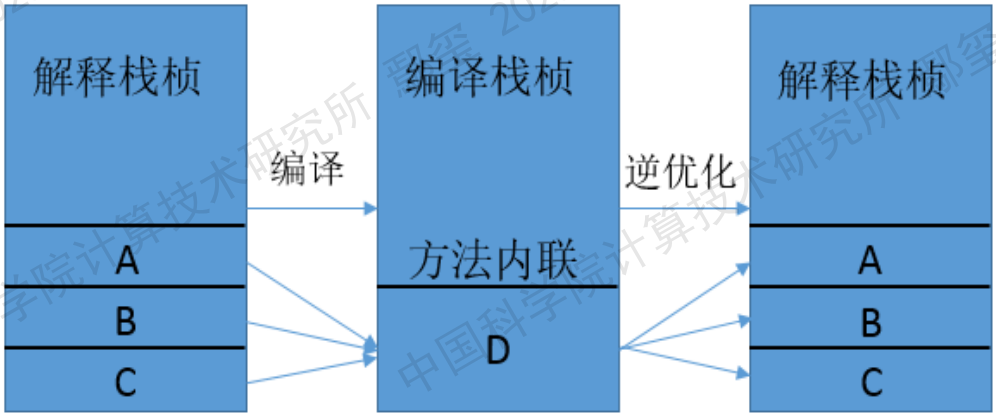


**图 2.10 逆优化时的栈帧变化**

**Figure 2.10 Stack frame changes during Deoptimization**

2. 方法内联技术[19]

方法内联技术指的是将被调用方法的代码直接编译到调用者代码当中，这样就省去了实际执行时的函数调用开销了。在程序执行过程中，函数调用占用的时间是比较大的，因为每次都要进行固定的保护以及恢复现场的操作。在 Java 这种函数调用比较多的语言中，这种弊端就体现的更加明显了，所以需要内联这种优化有效减少函数调用次数。但是 Java 当中常常遇到多态的情形，比如说 A 这个抽象类（接口的情形也是一样的）有 B、C、D 这三个具体实现类，它们都有 run 这个方法，在实际使用时，可能遇到的是 A.run()这样的方法调用形式，这时无法判断 A 的具体类型是哪一种，也就无法进行方法内联优化了。为了克服 Java 多态所带来的影响，Java 虚拟机中采用了基于类型分析的激进方法内联。还是上面这个例子，虽然 A 方法有三个子类，但是 B、C、D 这三个子类并不是同时加载到虚拟机中的，虚拟机遇到新的类型才会启动加载过程，当前如果只加载了 B 类，那么可以认为 A.run()一定调用的就是 B 中的 run 方法了，相应的内联优化也就可以顺利进行了，如果以后又加载了 C 类，那么之前的假设不成立了，基于只有 B 继承 A 这一假设而作出的内联编译也就要撤销，回退到解释执行的模式继续执行。

3. 分支预测技术[19]

分支预测技术指的是对 if/else 这样的分支语句的每一个分支子模块都进行计数统计，执行频次极低的分支可以认为从来没有执行到，可以不编译这部分代码，减轻实际方法编译的工作量，加速编译过程。这种优化方法基于这样的合理假设，在实际的编码过程当中，考虑到代码的健壮性，经常会有异常处理的分支，但是在实际执行时，程序发生异常的状况是很少的，异常处理的代码并不会被执行。对这部分进行编译，不仅浪费了编译资源，还占用了系统内存。不过要通过分支预测技术消除部分分支的编译，还要解决下面的两个技术问题。第一个是如何识别执行次数少的分支，第二个是识别次数少的分支一旦触发，如何及时发现。为了解决第一个问题，在虚拟机当中会统计每个分支指令的跳转方向，这样就得到了每个基本块实际执行的频率，对于频率接近 0 的基本块，就可以不编译了。对于第二个问题，在实际编译时，执行频率基本为 0 的基本块被编译成只有一个 uncommon trap 语句，一旦执行到这个基本块，立即通过这条语句触发如图 2.7

当中看到的逆优化流程，回退到解释器当中去执行。加入了分支预测技术优化之后编译出的 if/else 代码段可能如表 2.4 所示。

**表 2.4 分支预测产生的编译代码**

**Table 2.4 Compilation result of branch prediction**

```
if(foo != 0) {
} else {
    uncommon_trap() // <--- back to interpreter
}
```

可以看到在基于类型分析方法内联和分支预测技术当中，都用到了解释器作为激进优化失败后的“逃生门”，由此可以看到，虽然解释器的执行效率不高，但是正是因为解释器对程序正确执行的保障，使得即时编译器可以使用各种激进的优化方法，尽可以地提升程序执行效率。

4. 逃逸分析技术[19]

逃逸分析技术是 Java 虚拟机当中较新的优化技术，它并不是一种直接优化性能的手段，而是收集对象的相关信息，以供其他优化技术使用。逃逸分析的作用就是对 Java 对象的作用域范围进行分析。如果对象定义于某个方法中，并且被外部的方法所访问到，那么就叫作方法逃逸，如果对象可以被外部线程访问到的话，就称为线程逃逸。经过逃逸分析之后，如果发现某一个对象没有逃逸出方法或是线程外，就可以有针对性的采用一些高效的优化手段，如下所示。

栈上分配：传统的 Java 对象分配一定是在堆上进行的，通过持有对象的引用，可以访问堆中所存储在实例信息，垃圾收集器需要不时地标记堆中存活对象，清除已经无用的对象空间，并整理堆内存，这个过程耗费了不少时间。如果经过逃逸发现这个对象不会逃逸出方法之外，完全可以在栈上分配对象所需要的空间，对象所占用的空间会随着方法的栈帧出栈而自动销毁，不必特别的管理。在普通的应用中，局部对象占有不小的比例，如果使用栈上分配这种优化方法，可以减轻垃圾收集系统不小的压力。

同步消除：线程同步是一个非常耗时的操作，因为同步加锁会触发系统调用，涉及系统态与核心态的转换，虽然 JDK6[9]在虚拟机层面对 synchronized 关键词

进行了非常多的优化，但是不必要的加锁还是应该尽可能去除。如果逃逸分析得出一个变量并不会被其他线程所引用到，那么这个变量肯定是线程安全的，在这个变量上的加锁操作可以放心地去除。

标量替换：标量指的就是像 int、double 这样不能再分解的最基本数据类型，而 Java 的对象就是典型的包含多个标量的聚合量。根据实际情况，把一个 Java 对象分解成为多个标量来进行访问就叫做标量替换技术。如果逃逸分析结果显示一个对象不会发生线程逃逸并且可以被扩散的话，Java 虚拟机就不会真正的创建这个对象，转而创建多个标量分配于栈上或是寄存器当中。

以上介绍的是即时编译器中具有普适性的优化方法，各个平台的虚拟机都采用了这些优化，针对不同平台，比如在 MIPS 平台上，完全可以利用通用寄存器更多等优势实现一些平台相关的优化。

## 2.3 Java 虚拟机标准测试集

本文所用的测试集有 DaCapo[23]、SPECjvm98[24]这两个。它们都是现今为工业界与学术界认可的标准 Java 虚拟机性能测试集。通过不同的测试用例，测试内容覆盖了数值计算、内存访问、数据库事务、磁盘 I/O 等多个方面，全面地衡量了 Java 虚拟机在不同应用场景下的表现。这两个测试集的子测试程序分别如表 2.5 和表 2.6 所示。

**表 2.5 SPECjvm98 各子测试项内容**

**Table 2.5 Test items of SPECjvm98**

| 测试项目 | 测试内容 |
|---|---|
| compress | LZW 压缩算法 |
| jess | 交互式脚本系统 |
| db | 数据库访问 |
| javac | Java 源码编译器 |
| mpegaudio | 音频解码算法 |
| mtrt | 光线跟踪算法 |
| jack | Java 语法分析器自动生成器 |

**表 2.6 DaCapo 各子测试项内容**

**Table 2.6 Test items of DaCapo**

| 测试项目 | 测试内容 |
|---|---|
| avrora | AVR 微控制器模拟与分析程序 |
| batik | SVG 图像工具 |
| eclipse | Eclipse 测试程序 |
| fop | 格式转换工具 |
| h2 | 数据库引擎 |
| jython | Python 解释器 |
| luindex | 文档索引工具 |
| lusearch | 文档搜索工具 |
| pmd | Java 源码分析器 |
| sunflow | 图象渲染 |
| tomcat | Web 服务器 |
| tradebeans | 基于 Java Beans 的 daytrader 测试 |
| tradesoap | 基于 SOAP 的 daytrader 测试 |
| xalan | XML 转 HTML 工具 |

## 2.4 Java 虚拟机性能对比

本节使用业界广泛采用的 SPECjvm98 和 DaCapo 测试集来比 X86 和 MIPS 平台 Java 虚拟机的性能，所用的虚拟机版本号为 JDK8u77-b03[9]。

为了排除处理器性能对虚拟机的影响，本文首先测试了各处理器在同频率下的性能对比，测试集选用的是业界公认的 CPU 性能测试集 SPEC CPU2006[25]，实验中使用到的处理器平台参数如表 2.7 所示。

**表 2.7 X86 和 MIPS 平台处理器参数**

**Table 2.7 X86 and MIPS platform processor parameters**

| 架构 | CPU 型号 | 核心数 | 主频 | 内存 |
|---|---|---|---|---|
| X86 | i7-4500U | 2(超线程) | 1.8G | 8G |
| MIPS | 3A3000 | 4 | 1.4G | 8G |

其中龙芯 3A3000/3B3000 芯片是龙芯 3 号系列处理器最新产品[11]，于 2017 年流片成功，采用 28nm 设计工艺，主频可达 1.5GHz。其主要技术特征如下。

- 片内集成 4 个 GS464E 高性能处理器核；
- 片内集成 8MB 的分体共享三级 Cache；
- 通过目录协议维护多核及 I/O DMA 访问的 Cache 一致性；
- 片内集成 2 个 64 位带 ECC，667MHz 的 DDR2/3 控制器；
- 3B3000 片内集成 2 个 16 位 1.6GHz 的 HyperTransport 控制器（简称 HT）；
- 3A3000 片内 HT1 为 16 位 1.6GHz 的 HT 控制器，HT0 不可用；
- 每个 16 位的 HT 端口可以拆分成两个 8 路的 HT 端口使用；
- 片内集成 32 位 33MHz PCI(非标准电压)；
- 片内集成 1 个 LPC(非标准电压)、2 个 UART、1 个 SPI、16 路 GPIO 接口。

本文后续其他实验主要是在 3A3000 处理器上进行的。

图 2.11 展示了是 X86 和 MIPS 平台处理器的性能对比，实验结果以 3A3000 为基准进行了归一化处理。

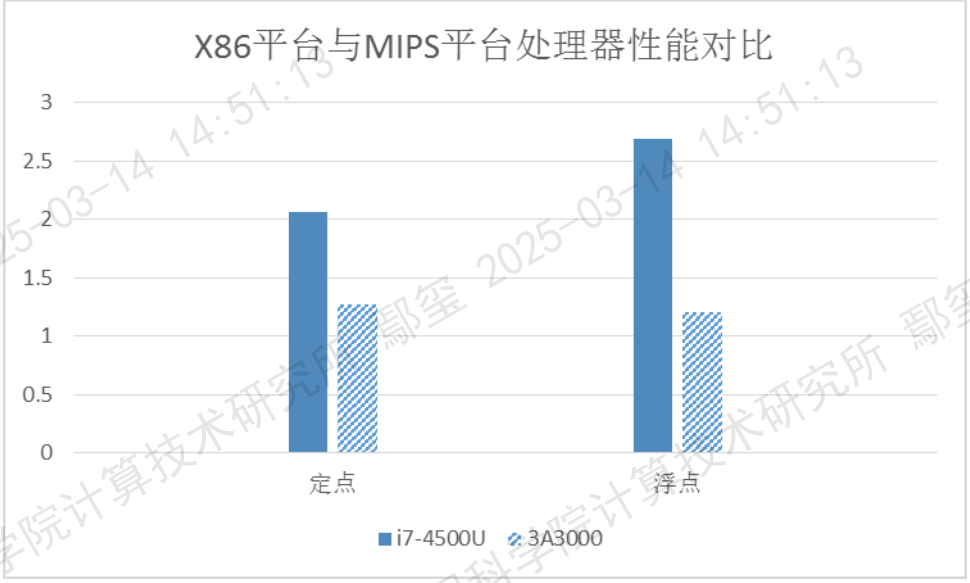


**图 2.11 X86 和 MIPS 平台处理器性能对比**

**Figure 2.11 X86 and MIPS platform processor performance comparison**

图 2.12 展示了各平台处理器频率均为 1GHz 时的虚拟机性能对比，实验结果以 MIPS 平台为基准进行了归一化处理。

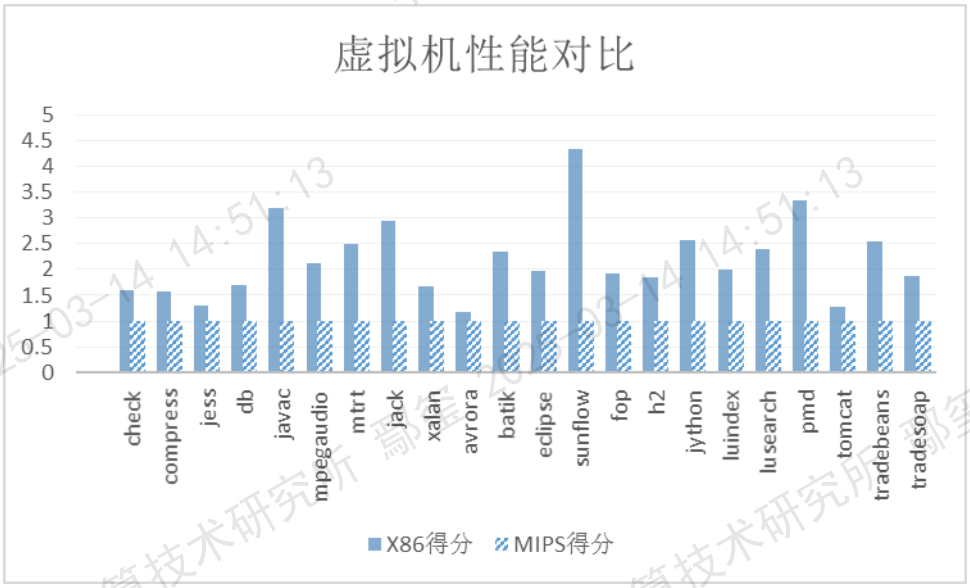


**图 2.12 X86 和 MIPS 平台虚拟机性能对比**

**Figure 2.12 Performance comparison of Java Virtual Machine on X86 and MIPS platform**

综合图 2.11 与图 2.12，可以看到，3A3000 平台与 i7-4500U 平台相比，虚拟机的性能差距远大于 CPU 的性能差距，比如 CPU 浮点性能差距为 1 倍的情况下 sunflow 测试项差距达到 3 倍多。由此可见，有必要针对 MIPS 平台的虚拟机进行性能分析和优化。

## 2.5 本章小结

本章对 Java 虚拟机的组成结构、相关优化技术及其性能评测程序进行了介绍。Java 虚拟机主要包括本地方法接口、执行引擎、垃圾收集器、类加载器和运行时数据区这五大基础模块。其中，执行引擎是执行 Java 程序的核心部件，由解释器与即时编译器组成，直接影响虚拟机的性能。在阐述了虚拟机组成结构与功能的基础上，又重点分析了解释器与即时编译器当中常见的优化技术。本章最后对比了当前 MIPS 平台虚拟机与 X86 平台虚拟机的性能差距，说明了 MIPS 平台上优化虚拟机性能的必要性。

中国科学院计算技术研究所 鄢銮 2025-03-14 14:51:13

# 第3章 分层编译的实现

分层编译[26]是从 OpenJDK7 开始引入的重要功能，通过解释器、C1 编译器和 C2 编译器之间的协调工作，虚拟机即可以获得很高峰值性能，又能快速启动，及时响应用户请求。由于 Java 虚拟机的平台兼容性问题，MIPS 平台 OpenJDK8 缺少对 C1 编译器的支持，导致无法开启分层编译的功能，应用程序启动性能存在严重瓶颈。本章通过实现 MIPS 平台 OpenJDK8 的 C1 编译器，完成了虚拟机的分层编译，大大提升了 MIPS 平台虚拟机的启动性能。

## 3.1 分层编译原理

通常，C2 编译器使用解释器来进行剖面分析（又称为 profiling，即收集方法相关信息），进而完成编译优化。 在分层编译中，除了解释器外，C1 编译器也可以在编译生成的代码当中加入 profling 功能。 由于编译代码运行速度比解释程序快，程序在性能分析阶段的效率更高。很多情况下，分层编译可以实现比单独 C1 编译器更快的启动，因为在应用程序初始化的早期阶段，C2 编译器生成的高度优化的代码已经可用。同时，分层编译也可以实现比单独 C2 编译器更好的峰值性能，因为更快的分析阶段允许更长时间的分析，这可能产生更好的优化。

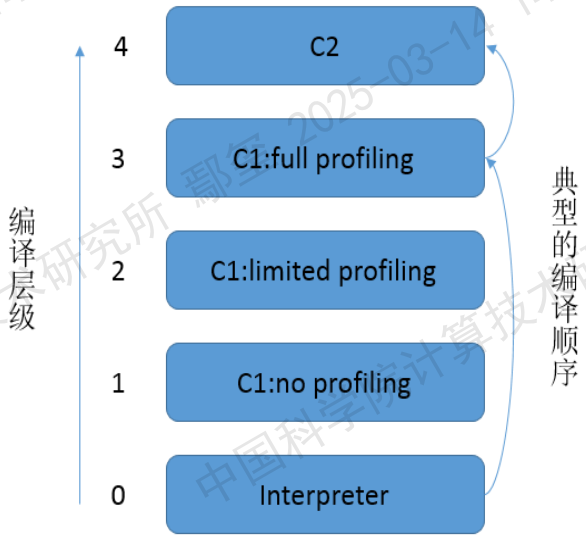


**图 3.1 Java 虚拟机的分层编译**

**Figure 3.1 Tiered Compilation in Java Virtual Machine**

图 3.1 展示了虚拟机当中的分层编译。级别 0 使用解释器执行 Java 字节码，由解释器来进行剖面分析，探测热点方法。热点方法会被安排到更高层级编译执行。级别 1 使用 C1 编译器编译方法，但是加入剖面分析功能，只有 C2 编译器也无法进一步优化的简单方法会被安排在级别 1 执行。层级 2 和 3 也是由 C1 编译器编译方法，但是层级 2 只收集部分方法信息，而层级 3 则会收集完整的信息。一般来说，层级 2 的生成代码运行效率比层级 3 要快 30%左右。级别 4 是由 C2 编译器来生成高度优化的方法代码。

Java 虚拟机中并不是所有层级都用到了的，图 3.1 当中的箭头展示了常见的执行级别转换，方法开始于解释器，C1 编译器编译生成收集方法信息的本地代码，C2 编译器生成最后的高度优化代码。

## 3.2 虚拟机的平台兼容性问题

当前 MIPS 平台可以运行的 Java 虚拟机是基于 OpenJDK 社区 JDK8u77-b03 版本移植的，本文研究工作开始时，MIPS 平台虚拟机的解释器部分与即时编译器中的 C2 编译器都已经完成移植工作，可以正常运行，但是并没有支持 C1 编译器，从图 3.1 中看，只能从 0 级向 4 级转换，依赖解释器对方法进行完全的剖面分析。无法开启分层编译的功能，导致虚拟机在 MIPS 平台上的性能受损。

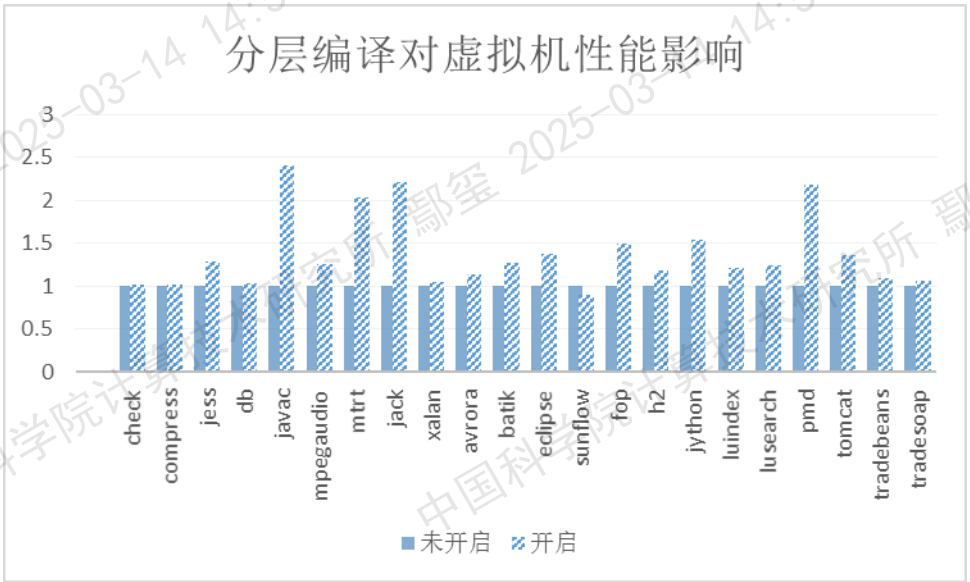


**图 3.2 X86 平台虚拟机分层编译性能提升**

**Figure 3.2 Performance improvement of Tiered Compilation in X86**

图 3.2 的实验结果展示了 X86 平台上分层编译对于程序性能的影响，测试集为 SPECjvm98 和 DaCapo，通过虚拟机参数-XX:TieredCompilation 来控制分层编译的开启与关闭。实验中使用了 X86 i7-4500U 处理器，主频 1.8GHz，内存 8G，操作系统为 64 位的 Ubuntu 16.04。

通过对实验结果分析，可以发现，分层编译使得 X86 平台上的 Java 虚拟机在各测试项上的性能均有所提升，最高提升幅度达到 120%。

由此推测，实现 Java 虚拟机在 MIPS 分支的分层编译功能，可以有效提升 MIPS 平台虚拟机的性能。

## 3.3 MIPS 平台 C1 编译器实现

### 3.3.1 C1 编译器的基本结构

在分层编译中，C1 编译器处在中间环节，过渡了从解释器直接到 C2 编译器的差异性，使得应用表现更流畅。C1 编译器设计目标是低启动时间与较小的内存占用，对方法编译经过三遍，每一遍之间程序都以中间表示形式存在。图 3.3 展示了 C1 编译器的基本结构。

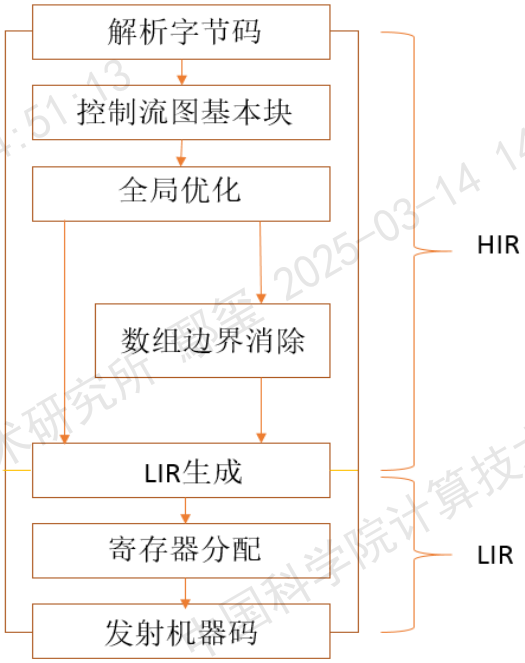


**图 3.3 C1 编译器基本结构**

**Figure 3.3 Structure of C1 compiler**

首先，被编译方法的高级中间表示形式（HIR，High-Level Intermediate Representation）通过对字节码的抽象解释来构建。它由一个控制流图组成，其基本块是单独链接的指令序列。 HIR 采用静态单赋值[27]（SSA，Static Single Assignment）形式，这表示对于每个变量，程序中只有一个点为其赋值。加载或计算值的指令既表示操作，表示操作返回值，所以操作数可以是先前计算出结果的指令。受益于 HIR 和 SSA 这种结构简单的表示形式，C1 编译器方便执行如常量折叠、值编号、方法内联、数组边界检查消除[28]和空检查消除等优化。

编译器的后端将优化的 HIR 转换为低级中间表示形式（LIR，Low-Level Intermediate Representation）。低级中间表示接近三操作数机器码，但还包括一些更高级别的指令，例如，用于对象分配和锁定的指令。虽然 LIR 的数据结构在所有目标平台之间共享，但是其中已经包含平台相关的代码部分。 LIR 重用了 HIR 的控制流，因此它不会改变基本块的结构。LIR 有利于各种低级优化，并且是寄存器分配器的输入，寄存器分配将虚拟寄存器映射到物理寄存器。

C1 编译器采用线性扫描寄存器分配算法[29]，这种算法的时间复杂度为 O(n)，其中 n 为虚拟寄存器的个数，相比于传统的时间复杂度为 O(n2)的图着色寄存器分配算法，它的运行时开销要小很多。

在寄存器分配完成之后，就可以以相当简单的方式生成机器码。编译器遍历 LIR 结构，把每条操作都翻译成相应的机器指令，并将这些机器指令发送到代码缓冲区当中。这个过程也会产生一些必要的调试信息。

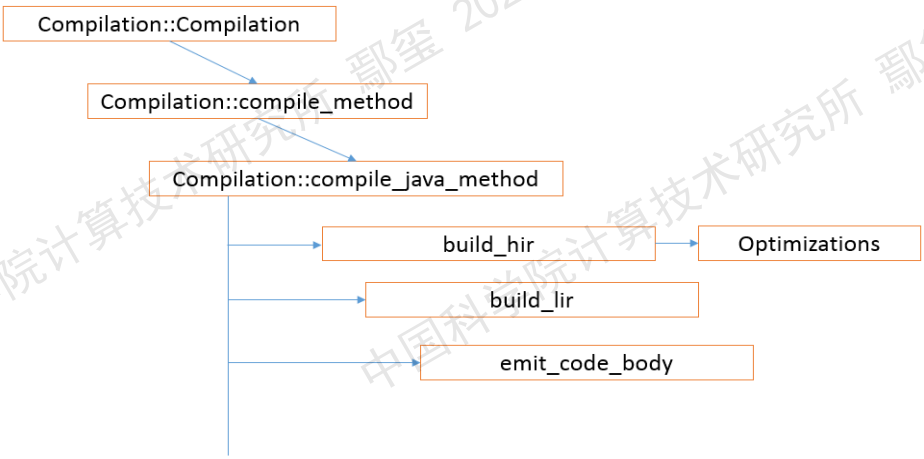


**图 3.4　C1 编译器代码调用路径**

**Figure 3.4　Process of C1 compiler**

实际 C1 编译器执行的时候，通过图 3.4 所示的调用路径，实现编译功能。compiler_java_method 方法当中的几个步骤与图 3.3 所示的一致，build_hir 方法在构建 HIR 结构的同时进行相关的优化工作，build_lir 方法在 HIR 基础上构建 LIR 结构并通过线性扫描算法分配物理寄存器，emit_code_body 方法将 LIR 结构转换为实际机器代码并发射到代码缓冲区中。

### 3.3.2 运行时数据区调整

从本节开始，对 C1 编译器实现过程当中的一些关键点进行说明。当前的 MIPS OpenJDK8 并没有开启 C1 编译器，其中包含的 C1 编译器代码还是从 JDK6 的 32 位平台遗留下来的。要将其成功移植到 JDK8 64 位平台上，首先要对内存空间管理的代码部分作出调整。

JDK8 相比于 JDK6，虚拟机对内存空间划分发生了改变[19]。Java 的字节码文件中存储了类的版本、字段、权限、方法、继承关系等描述信息，还有常量池这个关键的数据结构，用于存放编译器生成的各种符号引用和字面量。这些信息被称为类的元数据信息，在自定义类加载器比较少见的时候，这部分信息不经常发生变化，很少发生卸载，所以类元数据信息的存储区域被称为永久代，永久代通过参数-XX:MaxPermSize 来设定最大可用空间，当 Java 虚拟机加载的类的信息超过参数设定的值的时候，就会发生内存溢出的错误。从 JDK7 就开始了对永久代的移除工作，原先存储于永久代的数据部分转移到了 Java 的堆区或是本地内存当中。符号引用(Symbols)被转移到了本地内存而字面量(interned strings)和类的静态成员变量都被转移到了 Java 堆区域中。尽管如此，其他的数据还是存储在永久代的。

从 JDK8 开始，永久代的概念被完全移除，类的元数据信息（也叫做 Metadata 数据）被存储在叫做 Metaspace 的本地内存当中，可以通过-XX:MetaspaceSize 参数设置该空间的初始大小，在默认的情况下，Metaspace 的空间分配仅受到本地内存总量的限制。以前的虚拟机中的垃圾收集器处理存储在永久代中的类的元数据信息时需要专门的代码。由于 Metaspace 和 Java 堆具有相同的连续地址空间，因此 Metaspace 和 Java 堆可以无缝的管理，简化了垃圾收集器的设计。

内存空间管理的变化，导致了 JDK8 的 C1 编译器代码相比于 JDK6 发生了较大的变化，其中不少对于 Oop 对象的操作需要改成 Metadata，由此带来很多的代码重构工作。

### 3.3.3 LIR 汇编器实现

LIR 汇编器的作用是将 LIR 转换为本地机器代码，所以它与平台的关联度最高，是工作量最大的部分。在 Java 虚拟机中，LIR 的操作由 LIR_Op 基类及其子类来描述，它们的关系如图 3.5 所示。

Result 属性被置于 LIR_Op 这个基类当中，因为大部分操作都会有一个返回值。不同的操作种类是按操作数的个数来划分的。大部分的操作属于 LIR_Op0、LIR_Op1 或是 LIR_Op2，它们别代表有 0、1 或是 2 个输入的操作。图 3.5 当中最下面一层表示的是如函数调用、对象分配、类型检查这样的高级操作，这些操作随后会被转换为平台相关的机器指令。在一些操作当中可能会用到特定的寄存器，但是寄存器分配器会确保这些特定的寄存器的作用域限定在某个操作内，在操作前和操作后寄存器的值都是可以被覆盖的。

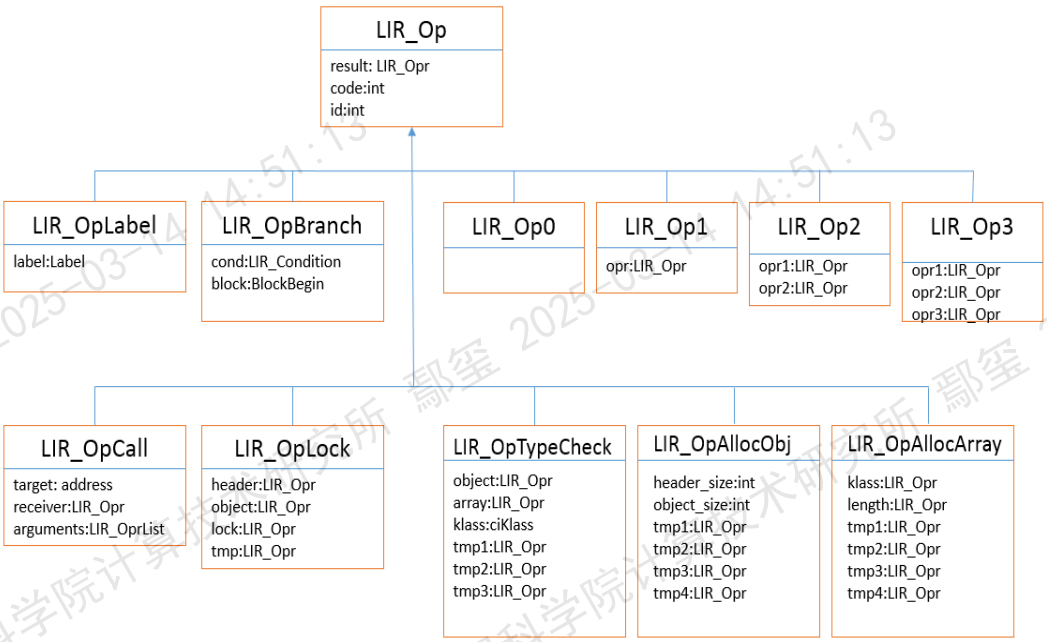


**图 3.5 LIR_Opr 类型继承关系图**

**Figure 3.5 Type inheritance diagram of LIR_Opr**

下面以 lir_add 的汇编代码生成为例说明 LIR 汇编器的实现逻辑。lir_add 在形式上属于图 3.5 当中 LIR_Op2 类的指令，它完成两个输入操作数的加法。为了方便说明，lir_add 的两个寄存器分别叫做左操作数与右操作数。

左操作数可能为单整型寄存器类型、双整型寄存器类型、栈类型或是地址类型，右操作数类型可能是常数类型、单整型寄存器类型、双整型寄存器类型。其中常数类型在 MIPS 上还需要继续区分是否为 16 位立即数，所以组合情况非常多。这里说明一下，双整型寄存器类型是 C1 编译器为 32 位 CPU 生成的，因为像 Long 型这样的数据在 32 位 CPU 上需要两个寄存器（低位寄存器与高位寄存器），在 64 位版本也沿用了双整型寄存器的规则，只是取数时直接从双整型寄存器的低位寄存器中取出 64 位数据即可。

如果左操作数为地址类型，右操作数为双整型寄存器类型，那么 lir_add 执行的是 Long 型的加法运算，它所生成的汇编代码就首先要从左操作数的地址当中取出操作数到一个临时寄存器当中，再从右操作数的双整型寄存器的低位寄存器中取出 64 位整型数据到另一个临时寄存器中，再执行两个寄存器的 64 位加法，最后将结果写回左操作数的地址当中。

### 3.3.4 适配器实现

适配器的功能是实现图 2.7 当中所示的解释执行与编译执行的转换。Java 虚拟机当中有两种类型的适配器，分别是从解释执行到编译执行的 i2c 适配器与从编译执行到解释执行的 c2i 适配器，如图 3.6 所示。

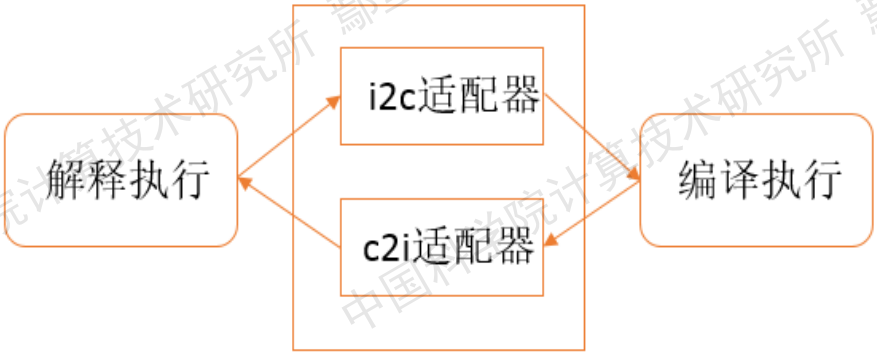


**图 3.6 适配器示意图**

**Figure 3.6 Adapter schematic**

转换过程之所以用到适配器，是因为解释执行与编译执行的栈帧布局并不相同，解释执行时，参数都是通过栈来传递的；编译执行时，由平台传参规则决定，比如在 MIPS 64 位平台上，前 8 个参数都是通过 A0-A7 这 8 个参数寄存器传递的，只有超出 8 个的部分才会通过栈进行传递。实际适配器实现时，首先要完成栈帧结构调整，再获取方法入口地址并跳转到该地址处执行。

## 3.4 错误调试实例

Java 虚拟机作为一个复杂的系统软件，核心代码行数超过 40 万行，其中 C1 编译器相关的代码在 4 万行左右，在移植过程当中涉及很多的底层原理，需要投入大量时间调试出现的问题。虚拟机的调试不同于一般应用层软件，存在以下难点。

- 出错原因涉及面较广。Java 虚拟机作为 Java 语言的运行时环境，设计精巧、功能复杂，与操作系统、编译器、C 语言类库都有所交互。任何一个环节出错，都可能导致虚拟机发生崩溃。
- 即时编译器的调试难度较高。与 C/C++这类静态编译语言不同，Java 语言的本地代码是由即时编译器在运行时产生的。动态生成的代码当中不方便加入打印语句，只能通过寄存器、内存镜像等信息查看出错现场，需要调试人员对虚拟机原理比较熟悉。
- 线程数目非常多。Java 虚拟机当中，解释执行线程、编译线程、管理线程、用户线程、GC 线程等大量线程在同时运行，可能会产生随机错误，问题复现周期长或是难以准确复现问题。

本节以 C1 移植过程中遇到的一个典型 Bug 为例说明调试 C1 编译器实现过程中的调试方法（调试过程中使用的是 Debug 版的虚拟机）。错误现象如表 3.1 所示（这是 Java 虚拟机发生崩溃时的输出，不必要的信息已经略去）。

**表 3.1 虚拟机错误信息**

**Table 3.1 Error information of Java Virtual Machine**

```
# A fatal error has been detected by the Java Runtime Environment:
# SIGBUS (0xa) at pc=0x000000ffe7fc6acc, pid=14859, tid=1099048563184
# Problematic frame:
#V[libjvm.so+0x65aacc]
CodeBuffer::finalize_oop_references(methodHandle)+0x234
```

根据错误信息，定位到 CodeBuffer::finalize_oop_references 函数的如表 3.2 所示的部分代码段。

**表 3.2 finalize_oop_references 函数部分代码**

**Table 3.2 Part code of function finalize_oop_references**

```
for (int n = (int) SECT_FIRST; n < (int) SECT_LIMIT; n++) {
    tty->print_cr("%d", n);
    // pull code out of each section
    CodeSection* cs = code_section(n);
    if (cs->is_empty())    continue;    // skip trivial section
    RelocIterator iter(cs);
    while (iter.next()) {
      iter.print();    //打印 iter 的信息
      if (iter.type() == relocInfo::metadata_type) {
        metadata_Relocation* md = iter.metadata_reloc();
        if (md->metadata_is_immediate()) {
          Metadata* m = md->metadata_value();
          if (oop_recorder()->is_real(m)) {
            if (m->is_methodData()) {        <---- error     SIGBUG
              m = ((MethodData*)m)->method();
            }
        if (m->is_method()) {
          m = ((Method*)m)->method_holder();
        }
```

进一步定位发现是由 m->is_methodData()引起的崩溃，通过添加打印语句发现 m 的值是 0x3442e61c3c0200ff，显然这是一个非法地址。在这段代码中，可以

看到，m 值来自于 md->metadata_value()语句，而 md 值又来自于 iter.metadata_reloc()这个重定位信息，通过添加打印语句 iter.print()。得到了如表 3.3 所示的 iter 信息。

**表 3.3　iter 的重定位信息**

**Table 3.3　Relocation information of iter**

| |
|---|
| @0x000000ff74028388: c05e<br>relocInfo@0x000000ff74028388 [type=12(metadata) addr=0x000000000083f3c8 offset=376] \| [metadata_addr=**0x000000000083f3c8 *= 0x3442ed5a3c0200ff** offset=0] metadata_value=0x3442ed5a3c0200ff: # |

可以看到，metadata_addr=0x000000000083f3c8 为重定位信息对应的生成代码位置，通过在添加-XX:PrintLIRWithAssembly 选项打印出来的信息当中，定位到了对应的 LIR 语句。如表 3.4 所示。

**表 3.4　出错位置处的 LIR 语句**

**Table 3.4　LIR code where the error happens**

| |
|---|
| 6 move [metadata:0x000000ffed5afd80\|M] [v0\|M]<br>**0x000000000083f3c8: lui v0,0xff <---- here**<br>0x000000000083f3cc: ori v0,v0,0xed5a<br>0x000000000083f3d0: dsll v0,v0,0x10<br>0x000000000083f3d4: ori v0,v0,0xfd80<br>8 move [Base:[v0\|M] Disp: 260\|I] [t1\|I]<br>0x000000000083f3d8: lw |

该出错位置处，在生成 move [metadata:0x000000ffed5afd80|M] [v0|M]这个 LIR 语句所对应的汇编代码，所以应该是 LIR 汇编器部分出现错误，汇编代码的生成位置在 c1_LIRAssembler_mips.cpp 文件的 LIR_Assembler::const2reg 函数处。所表 3.5 所示。

**表 3.5 const2reg 函数出错位置**

**Table 3.5 Position in function const2reg where the error happens**

```
void LIR_Assembler::const2reg(LIR_Opr src, LIR_Opr dest, LIR_PatchCode patch_code, CodeEmitInfo* info) {
    // ...
    case T_METADATA: {
      if (patch_code != lir_patch_none) {
        klass2reg_with_patching(dest->as_register(), info);
      } else {
        __ mov_metadata(dest->as_register(), c->as_metadata());
      }
```

重定位信息就是在 mov_metadata 函数当中写入的，参考 SPARC 平台，最终将 MIPS 平台的 mov_metadata 函数实现如表 3.6 所示，错误消失。

**表 3.6 mov_metadata 函数在 MIPS 平台上的实现**

**Table 3.6 Function mov_metadata on MIPS platform**

```
void MacroAssembler::mov_metadata(Address dst, Metadata* obj) {
  int index = oop_recorder()->allocate_metadata_index(obj);
  relocate(metadata_Relocation::spec(index));
  patchable_set48(AT, (intptr_t)obj);
  sd(AT, dst);
}
```

## 3.5 性能评测实验

实验使用了 SPECjvm98 测试集来度量 Java 虚拟机的启动性能。SPECjvm98 测试集各测试项具有运行时间短、启动快速的特点，能够代表客户端型应用程序，对虚拟机的启动性能进行测试。实验所用到的虚拟机版本号为 JDK8u77-b03，通过设置虚拟机参数“-Xms1G –Xmx1G”使得虚拟机堆大小固定在 1G，排除垃圾收集的影响。实验中使用到了 MIPS 兼容的龙芯 3A3000 四核处理器，主频 1400Mhz，内存 4G，操作系统为 64 位的 Fedora 21。

图 3.7 展示了虚拟机平均启动性能的提升，从图中可以看出，分层编译功能的加入使得 Java 虚拟机启动性能有了 33%的提升。

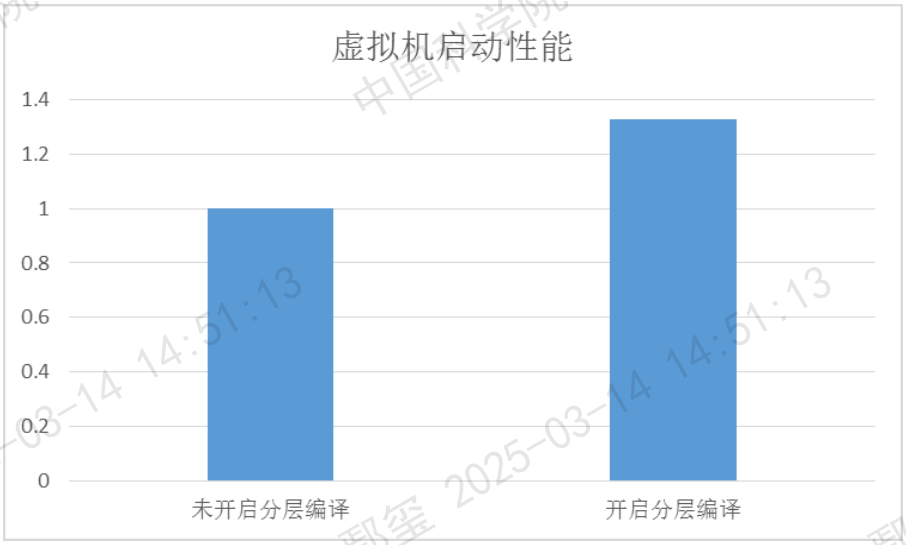


**图 3.7 SPECjvm98 测试集下分层编译带来的启动性能提升**

**Figure 3.7 Increase of the startup performance of SPECjvm98 due to Tiered Compilation**

图 3.8 展示了 SPECjvm98 各测试项的实验结果，可以看到，大多数测试项都是有性能提升的，其中 javac 测试项的性能提升幅度更是达到 113%。

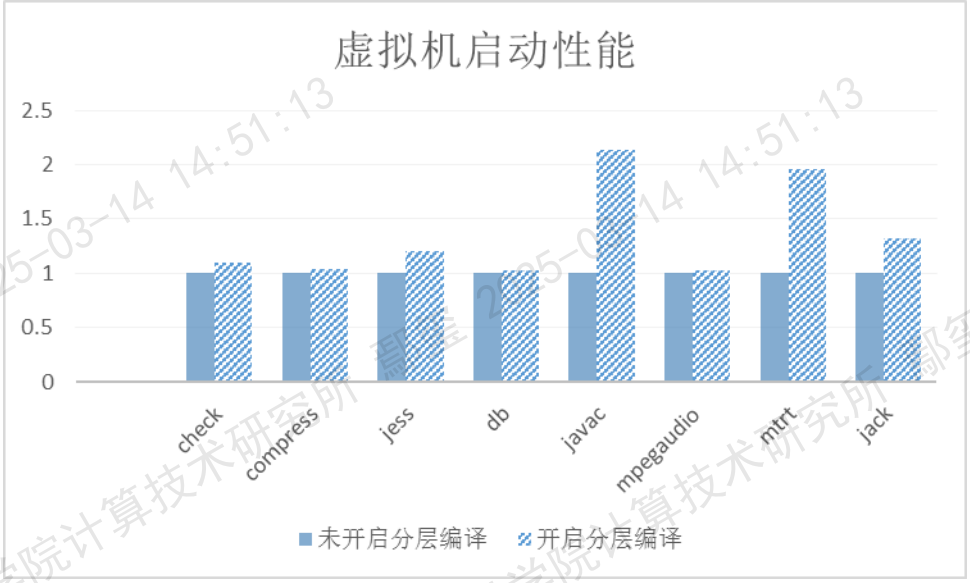


**图 3.8 SPECjvm98 测试集各测试项启动性能的提升**

**Figure 3.8 Increase of the startup performance of each test item in SPECjvm98**

通过与图 3.2 中 SPECjvm98 测试项的对比，能够发现，分层编译功能在 MIPS

平台上所带来的启动性能提升与 X86 基本一致，javac 和 mtrt 项目上的性能提升十分突出，可能是因为这两个测试项内部循环次数没有达到 C2 编译器的阈值，却达到了 C1 编译器的阈值，所以在 C1 编译器完成后，从原本的解释执行转化为了编译执行，性能有了大幅提升。

但是，mpegaudio 项目性能几乎没有提升，jack 项目提升幅度没有 X86 平台明显。造成这种异常的原因可能有以下两点。

- MIPS 平台 Java 虚拟机 C1 编译器的实现存在性能缺陷。当前 MIPS 平台的 C1 编译器可能有部分生成代码运行效率低下，而在某些测试项中又频繁地调用到了这些生成代码，导致应用程序启动性能并没有得到提升。
- 分层编译系统在 MIPS 平台的适配存在问题。虽然实现 C1 编译器后可以开启分层编译的功能，但是分层编译系统的很多参数设定还保持在默认状态，并没有针对 MIPS 平台进行适配，限制了分层编译的性能。

综上所述，分层编译功能可以有效提升 Java 虚拟机的启动性能，同时，MIPS 平台的分层编译系统还有进一步优化的空间。

## 3.6 本章小结

本章首先介绍了分层编译的原理，说明了解释器、C1 编译器和 C2 编译器的协同工作能有效提升了虚拟机的启动性能，并通过 X86 平台上的实验验证了分层编译对程序启动性能的提升效果明显。然后说明了 Java 虚拟机平台兼容性问题，MIPS 平台当前执行引擎只有解释器与 C2 编译器，无法开启分层编译导致应用程序性能存在严重瓶颈。接下来对 C1 编译器的原理与在 MIPS 平台的实现过程进行了阐述。最后通过实验验证了虚拟机性能提升。MIPS 平台分层编译的实现使得客户端型应用程序的性能提升了 33%，最高提升幅度达到 113%。

中国科学院计算技术研究所 鄢銮 2025-03-14 14:51:13

# 第4章 解释器性能优化

解释器负责 Java 字节码的解释执行，对应用程序的启动性能有很大的影响。解释器因为结构简单、易于维护、启动速度快等优点依然广泛应用在各种高级语言的虚拟机中。解释器的执行分为取指、指令分派、取操作数和执行这四个阶段，其中指令分派阶段在耗时较长，构成了解释器性能的瓶颈之一。本章在对解释器原理及其性能瓶颈分析的基础上，提出了一种超级字节码的方法来优化解释器的性能。

## 4.1 解释器工作原理

### 4.1.1 基于栈的指令集与基于寄存器的指令集

字节码文件当中所包含的指令流，本质上是基于栈式指令集架构[5]。指令集架构可以分为两种，一种是栈式指令集架构，一种是基于寄存器的指令集架构。基于栈的指令集当中大部分指令都是零地址指令，它们依赖于操作数据栈来获取操作数并存储操作结果，而基于寄存器的指令集架构依赖寄存器进行工作。现在一般计算机当中直接支持的指令集大都是基于寄存器的。下面简单说明这两种指令集的区别。

以最简单的计算“1+1”为例，如果使用基于栈式指令集的话，需要的指令如表 4.1 所示。

**表 4.1 字节码执行‘1+1’**

**Table 4.1 ‘1+1’ represented in bytecodes**

| |
|---|
| iconst_1 |
| iconst_1 |
| iadd |
| istore_0 |

开头的两条 iconst_1 指令依次把 1 存到栈当中，iadd 从栈当中的刚存入的 1 弹出，相加，并存回栈顶，最后的 istore_0 把栈顶内容存入局部变量表的第 0 个槽当中。如果是基于寄存器的指令集的话，以 MIPS 平台为例，完成同样的任务，

所需要的指令如表 4.2 所示。

**表 4.2 MIPS 汇编执行‘1+1’**

**Table 4.2 ‘1+1’ represented in MIPS Assembly**

| |
|---|
| daddi v0, r0, 1 |
| daddi v0, v0, 1 |

首先用 daddi 这条指令把立即数 1 加载到寄存器 v0 当中，然后再使用 daddi 指令将 v0 当中的内容和 1 相加，最后所得结果再存到 v0 当中。

从这个例子可以看出，基于栈的指令完成同样的内容，需要 4 条指令，而基于寄存器的指令只要 2 条，而且，从栈当中取数或者存数都是访问内存的操作，相比于访问寄存器来说非常耗时，所以基于寄存器架构的指令集在执行上要高效得多，这也是为什么大多数 CPU 都采用基于寄存器架构的指令集。但同时，基于栈的指令集架构也有它明显的优点，那就是实现简单，可移植性好。编译器生成指令的时候，不用考虑分配空间的问题，所需空间都在栈上操作。无论是在 X86 平台上还是在 MIPS 平台上，表 4.1 所示代码都可以执行，而表 4.2 的代码就不可能运行在 X86 平台上了。Java 虚拟机解释执行引擎所能识别的字节码就是基于栈式指令集架构所设计的。

### 4.1.2 Java 虚拟机的解释器

Hotspot 虚拟机当中有两种解释器，模板解释器和 C++解释器，但是模板解释器的实现更为高效，所以 MIPS 平台上默认使用模板解释器，下文中的解释器也都是指的模板解释器。

Hotspot 虚拟机内部的解释器框架图如图 4.1 所示。图中的序号表示了解释执行的过程。首先由栈帧当中的 bcp 指针确定方法的字节码存储位置，然后读取第一个字节码 iconst_1，由栈顶缓存寄存器状态和 iconst_1 分别决定分派表的行和列，由分派表表项中的查询结果获得 iconst_1 对应的本地机器码，并执行，这就是解释执行的过程。

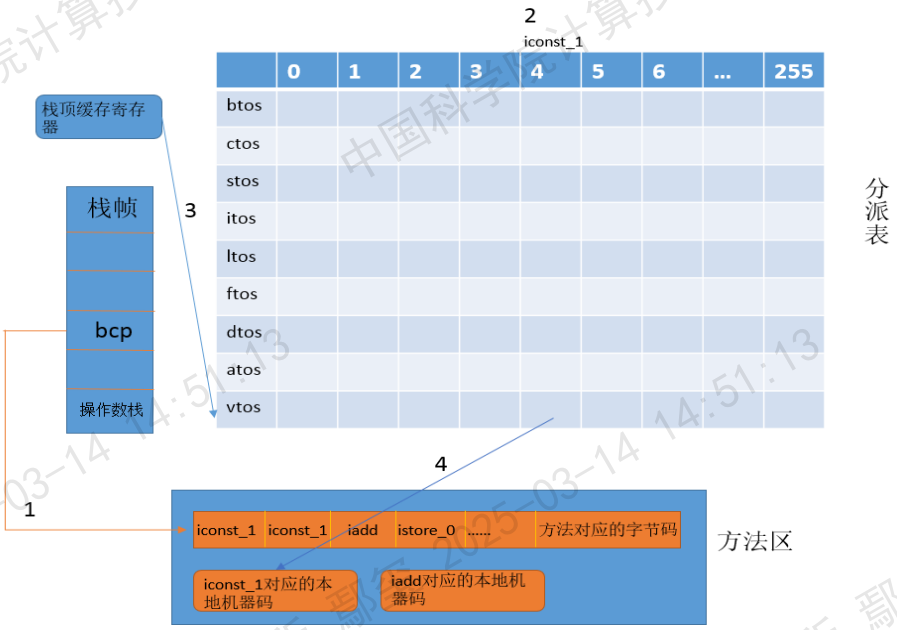


**图 4.1 解释器执行框架图**

**Figure 4.1 Interpreter execution framework**

下面对图 4.1 中的各组成部分进行说明。

1. 栈帧，又称为过程活动记录，是编译器用来实现函数调用过程的重要的数据结构。在程序执行过程当中，每一个调用函数会对应栈上的一个栈帧。Java 程序虽然是面向对象程序，但是实际执行的仍然是不同对象的方法，所以栈帧这种数据结构依然起着很重要的作用。Java 虚拟机当中有解释器栈帧、编译器栈帧、以及本地方法栈帧等不同类型的栈帧，它们对应不同的方法调用。解释器栈帧对应解释器执行 Java 方法时所用到的栈帧，编译器栈帧则是 JIT 即时编译时使用到的栈帧， 如果是从 Java 方法当中调用本地 C 语言方法，那就需要本地方法栈帧的支持了。MIPS 平台上 JDK8 的 Java 虚拟机解释器栈帧如图 4.2 所示，栈帧结构当中，局部变量表部分在最上面，分为 arg 与 local 两部分是因为 arg 部分是作为参数由调用者传入的。RA 表示返回地址，sender fp 和 sender sp 分别表示调用者的栈帧基址与栈顶，lvp 指向局部变量表的第一个变量，利用 lvp 这个首地址指针加上偏移量可以访问局部变量当中的每一个单元，method 指向当前解释执行的方法内存首地址，constant pool 指向运行时常量池，方法执行时所需要的常量就是通过这个指针获取的，bcp 一开始指向要执行方法的第一个字节码，随着方法的执行不断指向下一条字节码，这个指针就是在第 2 章中介绍 PC 了，stack

部分代表操作数栈，字节码就是依赖这个操作数栈进行计算的。

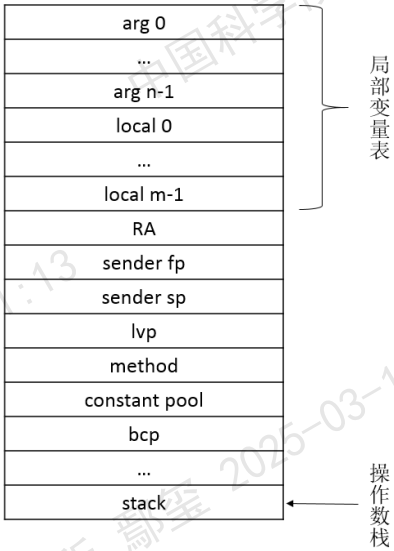


**图 4.2　解释器栈帧**

**Figure 4.2　Stack frame of Interpreter**

2. 方法区，这个内存区域不仅保存着对象的实例，还用于存储着每一条字节码对应的本地机器代码，Java 虚拟机为了尽可能提高执行效率，没有采用 C 语言来编写字节码的功能实现，而是直接写汇编代码并将其保存在方法区当中，实际运行的时候，跳转到相应的内存地址即可。

3. 分派表，这是解释器指令分派过程中用到的非常重要的数据结构。它是一个 9*256 的二维数组，如图 4.3 所示。

| | 0 | 1 | 2 | 3 | 4 | 5 | 6 | ... | 255 |
|---|---|---|---|---|---|---|---|---|---|
| btos | | | | | | | | | |
| ctos | | | | | | | | | |
| stos | | | | | | | | | |
| itos | | | | | | | | | |
| ltos | | | | | | | | | |
| ftos | | | | | | | | | |
| dtos | | | | | | | | | |
| atos | | | | | | | | | |
| vtos | | | | | | | | | |

**图 4.3　分派表结构**

**Figure 4.3　Dispatch Table**

在图 4.1 当中，也看到了分派表，表的每一列代表一个特定的字节码，每一行代表当前栈顶缓存寄存器的状态，各状态所代表的意义如表 4.3 所示。

**表 4.3 栈顶缓存寄存器状态说明**

**Table 4.3 Different states of stack caching**

| | 栈顶缓存寄存器状态 |
|---|---|
| btos | 一个 byte 型数据 |
| ctos | 一个 char 型数据 |
| stos | 一个 short 型数据 |
| itos | 一个 int 型数据 |
| ltos | 一个 long 型数据 |
| ftos | 一个 float 型数据 |
| dtos | 一个 double 型数据 |
| atos | 一个引用型数据 |
| vtos | 寄存器状态为空 |

从表 4.3 中可以看到，每条字节码本地机器代码入口地址可能有 9 个，在实际当中其实只有 2 个，具体的就由字节码类型来决定了，比如 iadd 字节码的入口地址可能是 itos 处，也可能是 vtos。

## 4.2 解释器性能瓶颈分析

从 4.1 节的介绍可知，每个字节码在解释执行时都需要通过指令分派找到本地代码的入口地址，相关研究也表明，指令分派过程构成了解释器的性能瓶颈[18]。以 isub 字节码为例，表 4.4 对比了该字节码在龙芯平台与 X86 平台生成的汇编代码，通过对比发现，字节码的本地代码组成结构在各个平台基本一致。

**表 4.4 isub 字节码对应的龙芯平台与 X86 平台汇编代码**

**Table 4.4 Assembly code of bytecode isub on MIPS and X86 platform**

| 龙芯平台 isub 指令汇编代码 | X86 平台 isub 指令汇编代码 |
|---|---|
| lw v0,0(sp) // **vtos** 入口处 | mov (%rsp),%eax //**vtos** 入口处 |
| daddiu sp,sp,8 | add $0x8,%rsp |
| lw v1,0(sp) // **itos** 入口处 | mov %eax,%edx //**itos** 入口处 |
| daddiu sp,sp,8 | mov (%rsp),%eax |
| subu v0,v1,v0 //实际计算部分 | add $0x8,%rsp |
| //以下都是寻找下一条字节码 | sub %edx,%eax //实际计算部分 |
| lbu s1,1(s0) | //以下都是寻找下一条字节码 |
| daddiu s0,s0,1 | movzbl 0x1(%r13),%ebx |
| dsll t2,s1,0x3 | inc %r13 |
| gsldx t3,0(t2,gp) | mov $0x7f60d2e4f5c0,%r10 |
| jr t3 | jmpq *(%r10,%rbx,8) |
| nop | add %al,(%rax) |
| nop | add %al,(%rax) |

以龙芯平台代码为例，可以看到 isub 字节码的组成分为入口、计算、指令分派这三部分。如果栈顶缓存寄存器为空，那么从 vtos 入口开始执行，否则从 itos 入口开始执行。实际完成减法功能的部分只有“subu v0, v1, v0”这一语句。功能代码部分结束之后便是找下一条字节码入口地址的指令分派代码。从静态代码上看，isub 字节码对应 12 条汇编代码，其中指令分派占 7 条，比例高达 58.3%；实际运行时，指令分派部分因为包含 jr 这类间接跳转指令，占比更高。像逻辑、算术运算等大部分字节码本身完成的功能都十分简单，主要耗时在指令分派，与 isub 情况类似。

## 4.3 超级字节码技术

### 4.3.1 技术原理介绍

针对字节码指令分派这一性能瓶颈点，本文提出一种超级字节码技术，即在虚拟机内部合成一对字节码，形成一个新的字节码，该字节码可以完成一对字节码的功能，但是减少了一次字节码之间的指令分派过程。如图 4.4 所示。

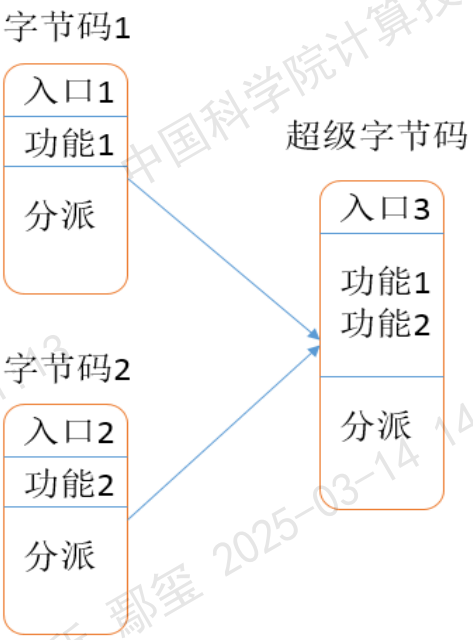


**图 4.4 字节码合成过程**

**Figure 4.4 Combination of two bytecodes**

这种技术实现的关键在于对字节码的判断与改写。

以 imul 与 iadd 这一对字节码合成为 fast_imadd 字节码的过程为例。要在 imul 字节码当中加入逻辑判断，如果 imul 字节码后面的字节码为 iadd，那么将当前的 imul 字节码改写为 fast_imadd，如果不是，把当前的 imul 字节码改写 fast_imul 字节码。fast_imadd 字节码完成乘加功能，fast_imul 完成乘法功能，与 imul 字节码一样，只是少了改写逻辑部分，这是为了防止每次遇到 imul 都要进行就否改写的判断，反而影响性能，所以每次合成字节码对实际上是添加了两条新的字节码。

### 4.3.2 合成字节码的选择

实际合成字节码对的时候，为了达到最优化效果，需要找到执行程序当中占比高的字节码对进行合成。虚拟机通过-XX:+PrintBytecodePairHistogram 参数可以打印出程序当中各字节码对的占比情况。本文首先统计了 SPECjvm98 和 DaCapo 测试集中各字节码对出现的频率，表 4.5 展示了部分高频字节码对及其在测试项当中的比例。综合字节码对在各测试项中频率分布情况，实际实验中合成了 fmul 和 fmadd、iconst_1 和 iadd 等 10 对字节码

**表 4.5　部分高频字节码在测试项当中的占比**

**Table 4.5　The proportion of some high-frequency bytecodes in the test items**

| 字节码 1 | 字节码 2 | 测试项 | 比例 |
|---|---|---|---|
| fmadd | fadd | mpegaudio | 3.001% |
| iinc | goto | check | 2.438% |
| dup | fast_igetfield | compress | 3.344% |
| iconst_1 | iadd | javac | 1.337% |
| aload_1 | iload_2 | tradesoap | 2.528% |
| aload_1 | fast_fgetfield | sunflow | 3.775% |
| fast_aload_0 | aload_1 | pmd | 1.514% |

## 4.4　性能评测实验

本文使用业界广泛采用的 SPECjvm98 和 DaCapo 测试集来度量 Java 虚拟机解释器性能优化实际效果。实验所用到的虚拟机版本号为 JDK8u77-b03，通过设置虚拟机参数“-Xms1G –Xmx1G”使得虚拟机堆大小固定在 1G，排除垃圾收集的影响。实验中使用到了 MIPS 兼容的龙芯 3A3000 四核处理器，主频 1400Mhz，内存 4G，操作系统为 64 位的 Fedora 21。

实验分为两组，分别测试优化前与优化后的解释器性能，每组实验进行 3 次取平均值。实验结果以优化前为基准进行了归一化处理。

解释器平均性能提升如图 4.5 示。可以看到，超级字节码的优化方案使得解释器的性能提升了 4.7%。

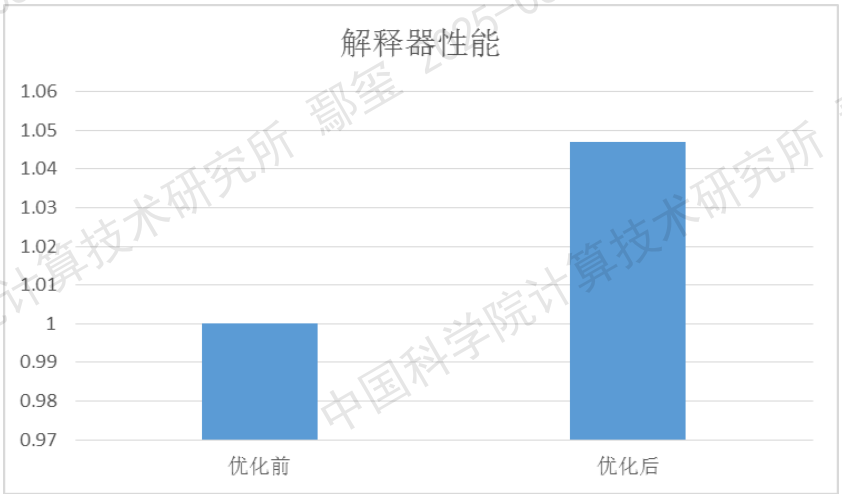


**图 4.5　SPECjvm98 和 DaCapo 测试集下解释器平均性能提升**

**Figure 4.5　Interpreter performance improvement in SPECjvm98 and DaCapo**

图 4.6 展示了各测试项的性能提升，从实验结果可以看到，大多数测试项优化项优化效果明显，比如 sunflow 的提升幅度高达 13.23%，mpegaudio 的提升幅度也有 12.54%。mpegaudio 和 sunflow 都是测试浮点运算程序，其中 fmul 和 fadd 组成的字节码对占比较多，均在 3%以上，所以将它们合成收益较大，同时 MIPS 平台 CPU 支持乘加指令，合成 fmul 与 fadd 字节码对将字节码功能部分指令也减少了。其他测试项的性能提升得益于多对字节码合成的整体效应。

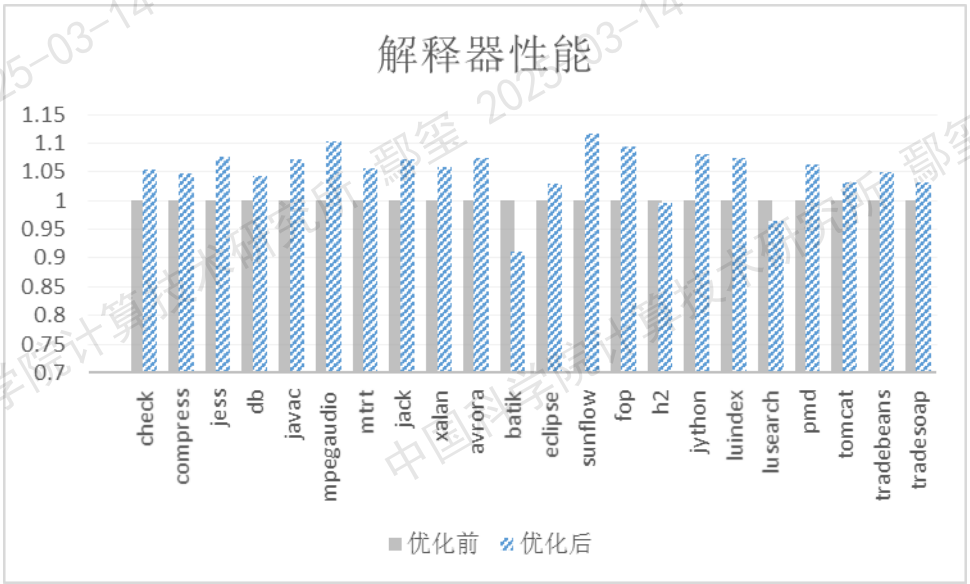


**图 4.6 SPECjvm98 和 DaCapo 测试项解释器性能**

**Figure 4.6 Interpreter performance changes in SPECjvm98 and DaCapo items**

当然，在图 4.6 中也看到了 batik、h2 和 lusearch 测试项性能有所下降，出现这种现象的可能原因有以下两种。

- 测试项中字节码合成失败的情况过多。比如在测试项当中以 fmul 开头的字节码对较多，但并不是 fmul 和 fadd 的组合。每一个 fmul 都要进行判断其后一条字节码是否为 fadd，如果不是就要改为 fast_fmul。在这类测试项中，合成判断过程占了较多运行时时间，却没有实际执行合成。
- 测试项中单个方法执行频率不高。对于字节码成功合成的测试项，也不一定会带来性能提升。因为字节码的合成是在方法执行过程当中进行的，所以如果方法执行频率较低的话，字节码合成的收益就不大。考虑一种极端情况，A 方法在程序中首次运行时对其字节码中的 fmul 和 fadd

等字节码对进行了合成，但是 A 方法之后没有再次执行了，这样一来，字节码合成工作就没有效果。

实验中只选取了 10 对字节码合成，一方面是考虑到了上述的负面情况，一方面是因为当前虚拟机已定义字节码数量达到 220 多个，可合成的字节码对数量也不多了。总体来看，超级字节码技术对于解释器的性能提升具有不错的效果。

## 4.5 本章小结

本章首先对解释执行与编译执行区别作了说明，然后介绍了栈式指令集和寄存器式指令集这两种不同的指令集设计，再详细阐述了 Java 虚拟机中解释器的工作原理，通过对字节码生成的本地代码的量化分析，指出解释器的指令分派阶段存在性能瓶颈。随后本文有针对性地提出了一种超级字节码技术，在虚拟机内部减少字节码的数量，达到优化解释器性能的目的。最后的实验部分表明，超级字节码技术使得解释器的性能平均提高 4.7%，最高提升幅度达到了 13.23%。本章的方法具有很好的通用性，也可以适用于其他平台虚拟机。

中国科学院计算技术研究所 鄢銮 2025-03-14 14:51:13

# 第5章 虚拟机参数调优

Java 虚拟机的调优工作一直是非常热点的话题，不同的启动参数，可能对程序性能影响很大，针对某个特定的应用场景，找到最优的参数组合是调优的目的。本文在 MIPS 平台的 Java 虚拟机基础上，选取部分启动参数，利用 OpenTuner[30] 自动调优框架进行调优，提升虚拟机在不同类型应用中的性能。

## 5.1 Java 虚拟机自动调优

### 5.1.1 自动调优介绍

近年来，程序自动调优越来越多地被运用在高性能计算和图形计算等领域，用以提升程序性能。自动调优取代了原先手动调优的大部分工作，程序员只要定义一下搜索空间与优化目标，就可以由程序自动调优了。相比于手动调优，程序自动调优有以下几个优点。

- 自动调优效率要远高于手动调优。随着应用程序越来越复杂，影响程序性能表现的参数也是多种多样，传统的依靠手工尝试各种参数组合的方式几乎是不可能的。
- 自动调优可移植性好。即使是同一个程序，也无法保证在不同的机器上用同样的参数组合可以达到最优性能。程序在新的机器上运行时，需要再次调优，这对于手工方式调优来说，极其耗费人力，而自动调优程序运行代价就要小的多了。
- 自动调优可以很容易实现在多个调优目标之间的平衡。程序调优的目标可以有多个，比如说性能、精确度、功耗和内存占用。有的程序就要求功耗低于特定值的时候性能最优。

### 5.1.2 Java 虚拟机启动参数介绍

Java 虚拟机的启动参数主要有以下两类[31]。

- 标准参数。这类参数在 Java 虚拟机当中功能稳定，各版本虚拟机中都不会改变，可通过 java –help 命令查看所有的标准参数。
- 非标准参数。非标准参数包括 X 参数与 XX 参数这两类。X 参数可以通

过 java –X 参数列出，不保证在所有版本虚拟机中都实现，常见的像-Xint 参数可以设置虚拟机以纯解释模式运行。XX 参数主要用于 Java 虚拟机的调优与优化，用来开启打印功能或是调整相关组件的运行时状态，比如-XX: CompileThreshold 用来设置方法调用计数器的阈值。

XX 类参数是 Java 虚拟机调优时的需要重点关注的参数，本文的调优工作就是针对这类参数进行的。根据 XX 参数的不同作用，可以按图 5.1 来进行划分。

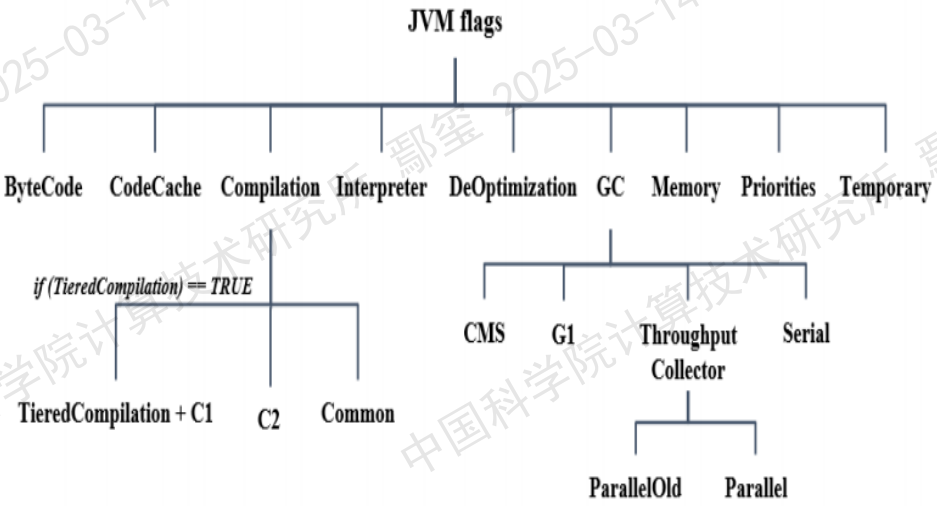


**图 5.1 Java 虚拟机参数分类**

**Figure 5.1 Parameter types of Java Virtual Machine**

Java 虚拟机中 XX 参数共有 600 多个，全部进行调优，搜索空间过大，实际不可行。编译的相关参数对即时编译器影响较大，因而是虚拟机参数调优的关键，本文选取了部分编译参数进行调优。

## 5.2 Java 虚拟机参数适配问题

Java 虚拟机要运行不同类型的 Java 应用程序，这些应用往往对启动时间、峰值性能、堆空间大小、垃圾收集时间等有着不同的要求，同一套虚拟机参数配置无法适应所有的应用场景，比如对启动时间有要求或是运行时间短的应用程序需要即时编译器较快生成编译后的本地代码，而要求峰值性能高的程序则可以让虚拟机的解释器或是 C1 编译器充分收集方法信息后再由 C2 编译器生成高度优化的本地代码。

早在 JDK5 时，MIPS 平台 Java 虚拟机的移植与优化工作就已经开始了[32]，经过十余年的努力，当前 MIPS 平台可以稳定运行的 Java 虚拟机为 JDK8 版本。一直以来，MIPS 平台 Java 虚拟机优化工作主要集中在虚拟机执行引擎与运行时环境方面，比如堆大小调整算法研究[33]、Java 线程控制结构首地址的优化[34]、方法调用优化[35]等，却忽略了对虚拟机内部参数的调优。

虽然 MIPS 平台 Java 虚拟机针对少量参数进行过手动调整，但是这种方式过分依赖于虚拟机开发人员的经验，且不能很好地适应不同的应用场景。

针对 Java 虚拟机的参数适配问题，本文首次提出在 MIPS 平台使用开源 OpenTuner 自动调优框架对虚拟机参数进行调优。

## 5.3 OpenTuner 框架介绍

OpenTuner 是一个新的开源自动调优框架，可以用来构建特定领域的、多目标的程序调优器。这个开源框架采用 Python 编写，不需要移植就可以在各平台运行。

OpenTuner 配置方便，它有很多的预定义的简单的数据类型，足够用来表示大多数问题了，当然，它也支持复杂的和用户自定义的数据类型，对不同的数据类型，会有不同的搜索方式。OpenTuner 内建类型如图 5.2 所示。

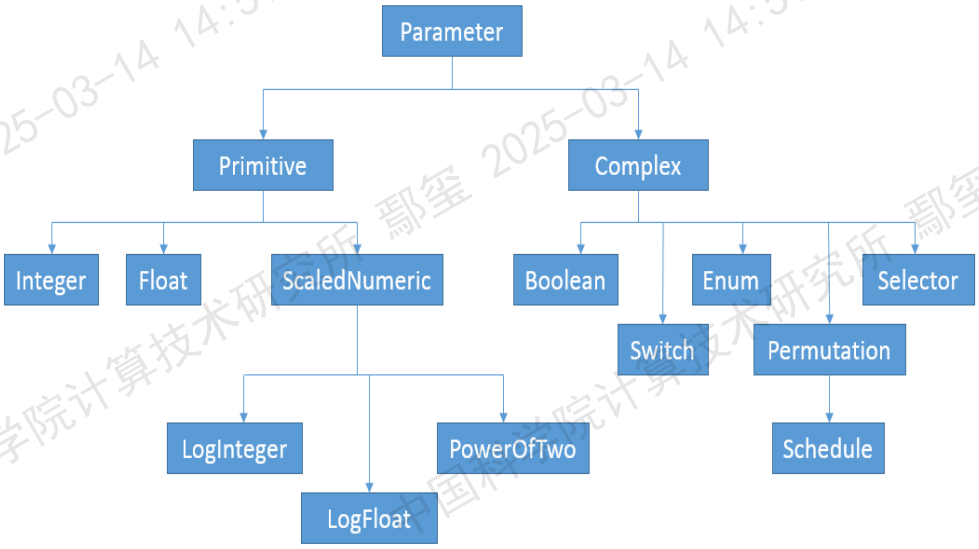


**图 5.2 OpenTuner 内建数据类型**

**Figure 5.2 Builtin types of OpenTuner**

可以看到，内建数据类型分为 Primitive 和 Complex 两类，它们为搜索引擎提供了不同的抽象。

- Primitive 类型包含的是数值型数据，这些数据都会有上界与下界，上界与下界视具体应用才能确定。图 5.1 中 Float 与 LogFloat 类型（Integer 与 LogInteger 也是相似的）都是用来表示浮点数据类型的，但是 Float 型的数值是直接提供给 OpenTuner 搜索引擎使用而 LogFloat 类型会进行 log 运算再供引擎使用。PowerOfTwo 类型数据则被限定取值只能是 2 的次方。
- Complex 类型是复杂数据类型，复杂数据类型支持用户自定义操作对其作出随机改变，适合于针对领域特定的问题来进行建模。从图 5.1 中可以看到，Boolean、Switch 和 Enum 都被归为了复杂数据类型，这是因为对于 Primitive 类型数据，都会遵循梯度变化，而这些数据的变化都是离散的、随机的，归为复杂类型考虑更合理。

除了提供多种数据类型用来配置外，OpenTuner 还内置了多种基于机器学习的搜索方法，使得应对不同类型的搜索空间时都可以快速地收敛到最优点，在实际执行的时候，不同的搜索方法都会被分配一定大小的测试集，同时进行搜索，它们的测试结果都保存在数据库当中。根据用户定义的调优目标实时评估结果的好坏，产生结果好的搜索方法会被分配更多的搜索任务，而结果差的则反之。

## 5.4 性能评测实验

本文使用业界广泛采用的 SPECjvm98 和 DaCapo 测试集来度量 Java 虚拟机参数调优效果，虚拟机版本号为 JDK8u77-b03。实验中使用到了 MIPS 兼容的龙芯 3A3000 四核处理器，主频 1400Mhz，内存 4G，操作系统为 64 位的 Fedora 21。

实验选取的 Java 虚拟机参数如表 5.1 所示，选取与编译有关的参数能使得调优收益较大，但同时从调优时间考虑只选择了部分编译参数。

**表 5.1 Java 虚拟机调优参数**

**Table 5.1 Parameters for tuning of Java Virtual Machine**

| 编号 | 虚拟机参数 | 作用 |
|---|---|---|
| 1 | DoEscapeAnalysis | 是否开启逃逸分析 |
| 2 | CompileThreshold | 方法调用阈值 |
| 3 | BackEdgeThreshold | 回边阈值 |
| 4 | OnStackReplacePercentage | OSR 比率 |
| 5 | InlineSmallCode | 内联方法大小 |
| 6 | MaxInlineLevel | 内联深度 |

实验分为两组，分别测试优化前与优化后的虚拟机性能，每组实验进行 3 次取平均值。实验结果以优化前为基准进行了归一化处理。其中 jython 和 eclipse 测试项因为在 OpenTuner 框架下无法正常运行而没有列出。

图 5.3 展示了虚拟机参数调优以后的性能提升效果，可以看到大部分测试项性能提升非常明显，javac、xalan、lusearch 和 pmd 测试项的提升比例较大，其中 lusearch 的提升幅度更是高达 86%，javac 测试项的提升效果也有 80%。

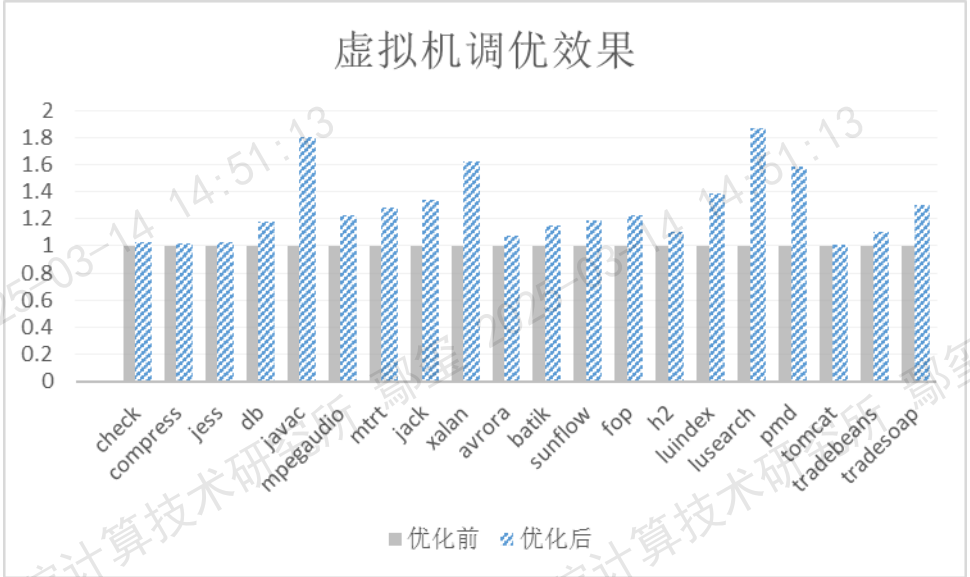


**图 5.3 SPECjvm98 和 DaCapo 测试项虚拟机调优测试**

**Figure 5.3 Tuning result of items in SPECjvm98 and DaCapo**

少数测试项如 check、compress、jess 和 tomcat 的优化效果有限或是没有优化效果。可能有以下原因。

- 测试项本身的可优化空间不大，比如 check 测试项在 MIPS 平台虚拟机上的运行时间仅为 0.1s 左右，其中大部分方法可能都还没有达到编译执行的阈值就已经结束了，导致调优效果不好。
- 参数的调优范围太小。从调优效率的角度出发，当前这些调优参数都是限制了在一定的范围内，比如 InlineSmallCode 对内联方法大小的限制为 1-6000 字节之间，如果测试项中对性能影响较大的方法本地代码大于 6000 字节，无法被内联，那么该测试项就对 InlineSmallCode 这个调优参数不敏感。
- 参数类型选择不适合。本文所选的调优参数都是与编译器有关的，可能部分测试项的瓶颈在其他因素上，比如说堆空间大小、CodeCache 大小、或是垃圾收集器的种类。

SPECjvm98 和 DaCapo 的每个测试项的调优结果都是一组参数值，在这组参数值下，对应的测试项性能最佳。本文随后选择调优性能提升最大的 luseacrch 测试项的参数配置来对所有测试项进行测试，Java 虚拟机平均性能有 13%的提升，如图 5.4 所示。

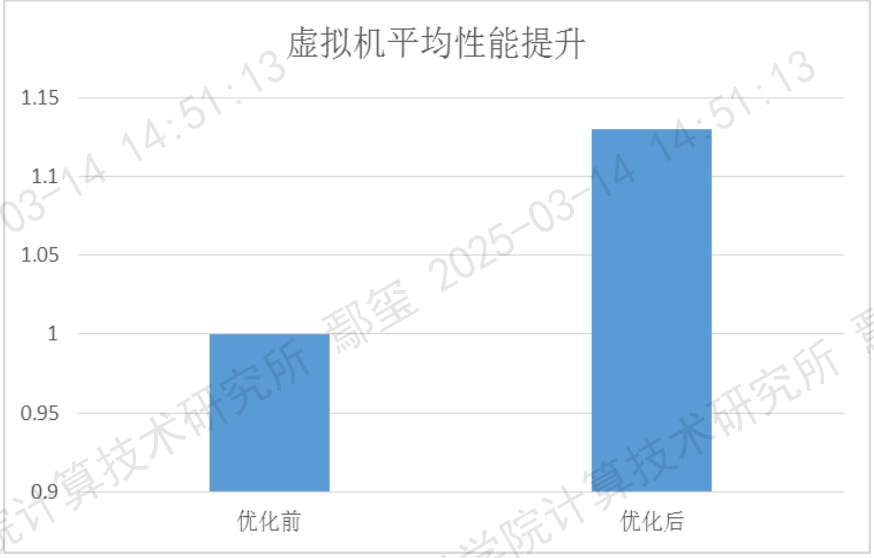


**图 5.4 SPECjvm98 和 DaCapo 测试集下虚拟机机平均性能提升**

**Figure 5.4 Java Virtual Machine performance improvement in SPECjvm98 and DaCapo**

表 5.2 展示了各测试项与具体参数值的对应关系，表中的 No.1 到 No.5 对应表 5.1 中的参数编号。

**表 5.2 SPECjvm98 和 DaCapo 各测试项最优启动参数**

**Table 5.2 Best parameters for items in SPECjvm98 and DaCapo**

| 测试项 | No.1 | No.2 | No.3 | No.4 | No.5 | No.6 |
|---|---|---|---|---|---|---|
| check | 关 | 4800 | 100000 | 140 | 5600 | 12 |
| compress | 开 | 5200 | 16000 | 140 | 6000 | 16 |
| jess | 关 | 4800 | 40000 | 100 | 2400 | 12 |
| db | 开 | 3600 | 24000 | 90 | 1600 | 20 |
| javac | 关 | 1600 | 12000 | 120 | 400 | 4 |
| mpegaudio | 开 | 1600 | 16000 | 140 | 1200 | 12 |
| mtrt | 开 | 800 | 76000 | 100 | 800 | 16 |
| jack | 开 | 2800 | 28000 | 80 | 1200 | 4 |
| xalan | 关 | 7600 | 12000 | 130 | 400 | 8 |
| avrora | 开 | 800 | 92000 | 40 | 800 | 8 |
| batik | 开 | 3600 | 8000 | 130 | 400 | 20 |
| sunflow | 开 | 800 | 56000 | 70 | 400 | 4 |
| fop | 关 | 2800 | 88000 | 120 | 400 | 16 |
| h2 | 开 | 4000 | 48000 | 140 | 2400 | 20 |
| luindex | 关 | 800 | 16000 | 40 | 400 | 20 |
| lusearch | 关 | 8000 | 76000 | 140 | 400 | 20 |
| pmd | 关 | 10000 | 28000 | 130 | 400 | 4 |
| tomcat | 开 | 4800 | 72000 | 140 | 800 | 4 |
| tradebeans | 关 | 2400 | 40000 | 120 | 400 | 12 |
| tradesoap | 关 | 3600 | 72000 | 120 | 400 | 20 |

图 5.5 则列出了按照 lusearch 性能最优的参数配置下，SPECjvm98 和 DaCapo 的每一个测试项的性能变化情况，可以看到，大部分测试项的性能依然是有所提升的。

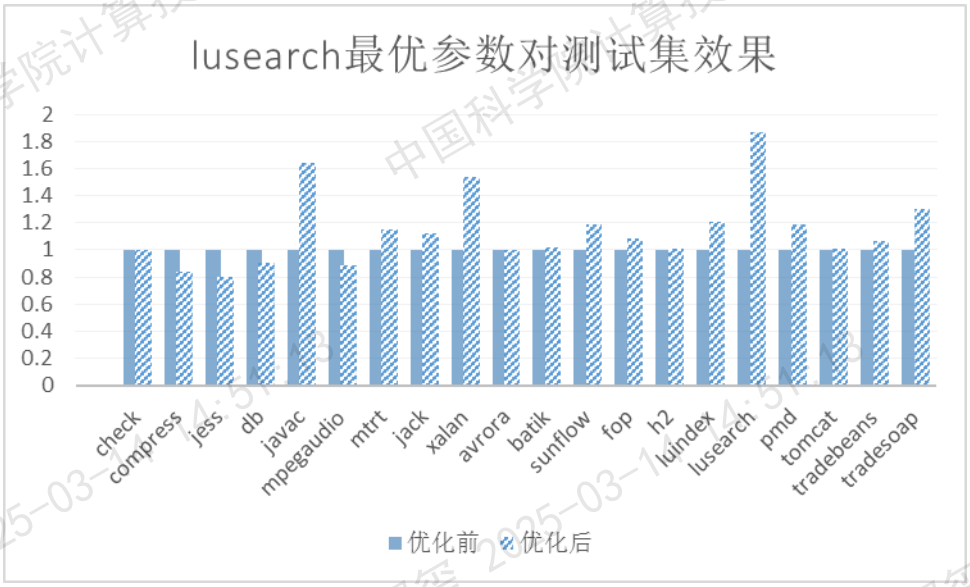


**图 5.5 SPECjvm98 和 DaCapo 测试集各测试项性能**

**Figure 5.5 Java Virtual Machine performance improvement in SPECjvm98 and DaCapo**

综上所述，MIPS 平台 Java 虚拟机调优具有非常好的效果，其中某些性能提升大的测试项参数配置对于 MIPS 平台整体参数调整有一定的指导意义。

## 5.5 本章小结

本章首先介绍了虚拟机自动调优在提升应用程序性能方面的重要作用，再阐述 Java 虚拟机当中各类启动参数及其作用，针对特定的应用场景，虚拟机当中的参数需要调整。本文使用了开源自动调优框架 OpenTuner 来进行自动调优。最后根据实验结果得出，自动调优有效提升了虚拟机在特定应用中的性能，最高幅度达到 86%。本章的调优方法具有通用性，在其他平台也可以采用。

# 第6章 总结与展望

Java 作为一种开发跨平台应用的面向对象的程序设计语言，因其通用性好、开发效率高和安全可靠等优点被广泛使用在 PC、Android 手机、互联网和嵌入式设备当中，并拥有庞大的开发者群体。随着 Java 应用复杂度的提升，对于 Java 应用程序的性能要求也越来越高，作为 Java 应用底层支持的 Java 虚拟机，直接影响到程序的执行效率。本文基于 MIPS 平台的 Java 虚拟机，分别在分层编译、解释器和启动参数调优等方面对虚拟机性能进行了优化。

## 6.1 论文工作总结

本文的主要工作内容有如下的三个方面。

第一、Java 虚拟机平台兼容性问题造成在非 X86 平台上的虚拟机可能缺失部分功能，无法充分发挥性能，限制了平台的应用推广。本文实现了 Java 虚拟机在 MIPS 平台的分层编译功能，分层编译能够对不同应用合理选择编译层级，经过实验验证，虚拟机启动性能提升达到了 33%。

第二、随着移动端 APP 应用的繁荣，Java 虚拟机的快速响应能力越来越重要。解释器的性能低下会造成应用启动缓慢的问题，本文分析了解释器的了性能瓶颈在指令分派阶段，并提出了一种超级字节码的方法，减少字节码数目，达到优化解释器性能的目的。经过实验验证，解释器平均性能提高了 4.7%。

第三，虚拟机参数调优。Java 虚拟机启动参数的配置会对应用的性能产生较大的影响，不同应用场景适合不同的启动参数，但是虚拟机启动参数种类繁多，取值区间大，导致手动找到一个合适的启动配置比较困难。本文使用了 OpenTuner 这个自动调优框架并选取了几个典型的对 Java 虚拟机编译参数，进行自动调优，使得在某些特定应用当中虚拟机性能最高有了 86%的提升。

本文部分研究成果将集成到龙芯 OpenJDK8 版本库当中，并投入实际生产。

## 6.2 未来研究方向

本文的研究工作不足之处主要体现在以下两个方面。

第一，分层编译没有进行优化，C1 编译器实现以后，MIPS 平台就可以开启分层编译功能，利用 C1 和 C2 编译器的特性，对于特定的应用，有针对的采用相应编译层级。当前分层编译功能开启以后，各参数都还是原始默认值，比较粗糙，没有利用 RISC 架构的优势进行优化。

第二，调优时间过长，当前 MIPS 平台上利用 OpenTuner 对虚拟机参数进行调优时，选取 6 个调优参数，1min 运行时长的程序需要调优 24h 后才能找到最优参数配置。这样的效率无法对实际运行时间较长的程序进行调优。

针对上述问题，可以有进一步的研究方向。

第一，动态编译系统的优化。结合 MIPS 平台的特性，对编译系统进行优化。针对热点识别、编译阈值、编译队列长度、0 到 4 级的执行层次如何选择这些问题，可以结合 OpenTuner 自动调优框架对编译系统进行深度调优，也可以提出新的数学模型来解决。

第二，调优效率的优化。针对调优时间过长的问题，可以构建分布式调优框架，利用多台机器同时工作来缩短调优时间，也可以对调优搜索算法进行深入研究，针对某个特定的问题，构造快速搜索算法来加快调优过程。

中国科学院计算技术研究所 鄢銮 2025-03-14 14:51:13

# 参考文献


[1] James Gosling, Bill Joy, Guy L. Steele, Java Language Specification[S], Addison-Wesley Longman Publishing Co., Inc. Boston, MA, 1996.

[2] Weiwu Hu, Yifu Zhang, and Jie Fu. An introduction to cpu and dsp design in china. Science China Information Sciences, 59(1):1–8, 2016.

[3] Tim Lindholm, Frank Yellin. Java Virtual Machine Specification[M], Addison-Wesley Longman Publishing Co., Inc., Boston, MA, 1999

[4] TIOBE. TIOBE 编程语言排行榜. https://www.tiobe.com/tiobe-index.

[5] Yunhe Shi, Kevin Casey, M. Anton Ertl, and David Gregg. Virtual machine showdown: Stack versus registers[C]. ACM Trans. Archit. Code Optim, 4(4):2:1-2:36, January 2008.

[6] Thomas Kotzmann, Christian wimmer, Hanspeter Mossenbock, Thomas Rodriguez, Kenneth Russell, David Cox. Design of the Java HotSpotTM Client Compiler for Java 6[C]. ACM Transactions on Architecture and Code Optimization, Vol.5, May 2008.

[7] T. Suganuma, T. Yasue, M. Kawahito, H. Komatsu, T. Nakatani. A dynamic optimization framework for a Java just-in-time compiler[C]. In ACM Coference, Oct 2001. pp. 185-190.

[8] Michael Paleczny, Christopher Vick, Cliff Click. The Java HotSpotTM Server Compiler[C]. Proceedings of the 2001 Symposium on JavaTM Virtual Machine Research and Technology Symposium, April 2001.

[9] OpenJDK [EB/OL]. http://openjdk.java.net/.

[10] （英）Dominic Sweetman 著，李鹏，鲍峥，石洋等译. MIPS 体系结构透视[M]，机械工业出版社, 2008.

[11] 龙芯 3A3000/3B3000 处理器用户手册（上册）. 龙芯中科技术有限公司, 2017.

[12] 龙芯官方网站[EB/OL]. http://www.loongson.cn/loongson/.

[13] David Detlefs , Christine Flood , Steve Heller , Tony Printezis, Garbage-first garbage collection[C]. Proceedings of the 4th international symposium on Memory management, October 24-25, 2004, Vancouver, BC, Canada.

[14] Sheng Liang, The Java Native Interface: Programmer’s Guide and Specification [S], 1999.

[15] BJones, R. Lins, Garbage Collection: Algorithms for Automatic Dynamic Memory Management[C], New York, 1996.

[16] Thomas Kotzmann, Christian wimmer, Hanspeter Mossenbock, Thomas Rodriguez, Kenneth Russell, David Cox. Design of the Java HotSpotTM Client Compiler for Java 6[C]. ACM Transactions on Architecture and Code Optimization, Vol.5, May 2008.

[17] James R. Bell. Threaded code[C]. Commun. ACM, 16(6):370-372, June 1973.

[18] M. Anton Ertl and David Gregg. The structure and performance of efficient interpreters. Journal of Instruction-Level Parallelism, 5:2003, 2003.

[19] 周志明. 深入理解 Java 虚拟机：JVM 高级特性与最佳实践. 北京：机械工业出版社, 2014.

[20] SPEC jvm2008 [J/OL]. http://www.spec.org/jvm2008/.

[21] Fink, S.J, Qian, F. Design, implementation and evaluation of adaptive recompilation with on-stack replacement[C]. Preceeding of the International Symposium on Code Generation and Optimization. IEEE Computer Society, CA, 2003. pp. 241-245.

[22] Urs Hölzle , Craig Chambers , David Ungar. Debugging optimized code with dynamic deoptimization[C]. Proceedings of the ACM SIGPLAN 1992 conference on Programming language design and implementation, p.32-43, June 15-19, 1992, San Francisco, California, USA.

[23] SPEC jvm98 [J/OL]. http://www.spec.org/jvm2008/.

[24] DaCapo[J/OL]. http://www.dacapobench.org/.

[25] SPEC CPU2006 [J/OL]. http://www.spec.org/cpu2006/.

[26] Gilbert Joseph Hansen. Adaptive Systems for the Dynamic Run-time Optimization of Programs. PhD thesis, Pittsburgh, PA, USA, 1974. AAI7420364.

[27] Ron Cytron, Jeane Ferrante, Barry K. Rosen, and Mark N. Wegman: Efficiently Computing Static Single Assignment Form and the Control Dependence Graph[C]. ACM TOPLAS, October 1991. pp. 451-480.

[28] Wurthinger, T., Wimmer, C., Mossenbock, H. Array bounds check elimination for the Java HotSpotTM client compiler[C]. Proceedings of the International Conference on Principles and Practice of Programming in Java. ACM Press, New York, 2007.

[29] Christian Wimmer. Linear Scan Register Allocation for the Java HotSpot Client Compiler. A thesis submitted in partial satisfaction of the requirements for the degree of Master of Science(Diplom-Ingenieur) Institute for System Software Johannes Kepler University Linz, August 2004.

[30] J. Ansel, S. Kamil, K. Veeramachaneni, J. Ragan-Kelley, J. Bosboom, U.-M. O'Reilly, S. Amarasinghe. Opentuner: An extensible framework for program autotuning[C]. International Conference on Parallel Architectures and Compilation Techniques, August 2014.

[31] Oracle 官方网站[EB/OL]. http://www.oracle.com/technetwork/articles/java/.

[32] 刘超. 基于龙芯处理器的 Java 运行环境的移植与优化[D]. 北京：中科院计算所，2006.

[33] 杨勇强. 基于四核龙芯 3 号平台的 HotSpot6 移植与优化[D]. 北京：中科院计算所，2012.

[34] 武慧. 龙芯 MIPS 64 位 Java 虚拟机的实现与优化[D]. 北京：中科院计算所，2013.

[35] 王一光. MIPS 64 平台下 JVM 性能优化研究与实现[D]. 北京：中科院计算所，2015.

# 致 谢

在本文相关研究工作期间，许多老师、同学和员工无私地给予了我很多帮助和指导。在此，我谨向曾经关心和帮助我的人表示由衷的感谢。

首先，感谢敖琪与傅杰两位师兄，从我进入 Java 组到即将毕业的这段时间里，无论是在生活上还是学习上，都提供了无私的帮助。两位师兄敏锐的洞察能力与严谨的工作态度给我树立了好的榜样，每次在研究过程当中遇到困难的时候，师兄都会耐心的讲解。论文的顺利完成，与两位师兄的悉心指导是分不开的。

感谢我的导师齐子初老师，在我硕士研究生期间在生活与学业上都提供了无私的指导，使得我很快地了解了研究生期间所要学习的知识，从而能够很好地融入龙芯 Java 组的研究工作当中。

感谢肖俊华老师，肖老师深厚的学术素养与严谨的治学态度值得每个人去学习，同时，在研究生期间，肖老师不辞辛劳，每周都组织论文研讨，这让我很快的学会了如何去阅读总结论文。

感谢 Java 组的姜韶峰、李方媛、王雪、孙亚奇、黄旭光和翟翔，大家互助帮助，共同进步。Java 组就像一个大家庭，能够让人感到温暖。

感谢李环宇、王鹏师兄和王韩月师姐，你们无私地传授在找工作期间获得的经验与教训，使得我在找工作时少踩了许多坑。

感谢王俊儒、赵士鹏、吴畏等师弟和师妹们，你们乐观积极的生活态度与严肃认真的科研精神感染了我，让我对生活与学习都充满热情。

感谢张立志、李策、汪浪等 2015 届微处理器研究中心的全体同学，大家彼此帮助、共同奋斗、一起进步，一起度过了三年的研究生时光，能够认识你们这样优秀的伙伴，我感到非常荣幸。

感谢陈飞、崔浩然、陈旭等在怀柔期间学习时的室友，虽然只在一起相处了一年的时光，但还是留下了美好难忘的回忆。

最后，感谢我的亲人和朋友，这么多年来一直在我在身边，关心着我，给予我力量，感激之情无以言表。

2018 年 6 月

# 作者简历及攻读学位期间发表的学术论文与研究成果

**作者简历：**

2011 年 9 月——2015 年 7 月，在北京理工大学软件学院获得学士学位。

2015 年 9 月——2018 年 6 月，在中国科学院计算技术研究所攻读硕士学位。

获奖情况：

工作经历：

**已发表（或正式接受）的学术论文：**

**申请或已获得的专利：**

**参加的研究项目及获奖情况：**

中国科学院计算技术研究所 鄢玺 2025-03-14 14:51:13